SharpeArena: An Agentic Point-in-Time (PIT) RL Environment for Trading

Tiberiu Toca
General Liquidity
tibi.toca@gmail.com

Abstract

Reinforcement-learning results in trading are produced by the party being evaluated, on an environment that party controls and defends as realistic on the strength of its frictions: transaction costs, slippage, position limits. Frictions constrain the agent, and say nothing about whether the simulated world behaves like a market or about whether the trajectory the environment emitted is the one it ran. We present SharpeArena, an agentic point-in-time environment that forecloses three failures rather than discouraging them: no data-layer operation returns a bar past the engine's cursor, golden fingerprints pin the Rust core and its native, WebAssembly and Python surfaces each assert them, and runs record decisions only, so a doctored trajectory replays to something else. Agents are external programs, language models included, that reach the engine through a governed JSON contract. A host-side parser enforces episode semantics, and a transport gate turns every recorded wire fault into a typed failed cell: an empty decision against a stateful engine is a true hold, so a wedged agent would otherwise ride its last position to the end of the window and emit a return series indistinguishable from a deliberately conservative one. Around that loop the environment adds a strategy-generation path over a closed non-executable candidate language, whose raw candidates are counted by the host and bound to a falsifiability manifest before validation, and a forward paper-trading arm whose evidence is kept apart from deterministic replay and carries no replay guarantee. Scoring is delegated to the separate SharpeBench kernel. We calibrate the environment's own diagnostics with fixed reference policies: nothing is trained and no result is a claim about markets. We find that the canonical generator fails its own realism diagnostic on 23 of 24 seeded panels, and that a predeclared sweep of its volatility-clustering knobs finds no Calm setting that survives. We also find no reference policy rank-eligible, and a single-seed reading of the ecology probe does not survive replication. We offer a producer whose guarantees are properties of committed code rather than of its authors' good faith, and diagnostics that report where its tape is not yet market-like.

1 Introduction

Every published number in agent trading evaluation is produced by the party being evaluated, on an environment that party controls. Three failures upstream of scoring void any score, and none of them is visible in the score itself. An agent that observes one future bar has an edge no deflation can remove; look-ahead bugs are a pervasive and usually invisible failure mode of backtests, alongside the selection bias the deflation literature centers [Bailey and de Prado, 2014]. An environment that is not bit-reproducible cannot support a leaderboard, because the same submission scores differently

Preprint.

on different machines and no one can check anyone else's number. And a trajectory that cannot be recomputed from the environment that allegedly produced it is a self-reported claim, not evidence.

The trading environments agents are trained and evaluated on close none of the three, and the realism they do claim is claimed on a different axis. FinRL states that it replicates the live stock market and cashes that statement out entirely as frictions: transaction cost, liquidity and risk aversion [Liu et al., 2020]. TradeMaster likewise incorporates practical constraints such as transaction costs, slippage and a non-negative balance in the name of a more realistic environment, defers its transition function to FinRL-Meta by citation, and reports backtest metrics where a validation of the simulated market would go [Sun et al., 2023]. ABIDES claims high fidelity in its title and asserts that its platform produces realistic slippage naturally, without quantitative evidence for either [Byrd et al., 2020]; ABIDES-Gym inherits those background-agent configurations without revalidating them and acknowledges only rare unrealistic behavior in the synthetic market [Amrouni et al., 2021]. FinRL-Meta is the honest exception: it names the sim-to-real gap and enumerates what it leaves unmodelled, including permanent market impact of limit orders and a non-resilient limit order book, though it quantifies none of it and tests no distributional property of returns [Liu et al., 2022]. mbt_gym makes no realism claim at all; it claims correctness and demonstrates it by recovering a known closed-form optimum [Jerome et al., 2023], which is verification rather than validation, and is honest precisely because it never conflates the two.

Read together, the pattern is a substitution: frictions stand in for dynamics. A friction bounds what the agent is allowed to do. It carries no information about whether the process the agent acts on behaves like a market, and none at all about whether the trajectory the environment reports is the one it ran. SharpeArena addresses both halves of what the substitution leaves out: the three producer failures above are excluded structurally, and the question of whether its own tape is market-like is asked as a committed diagnostic that the canonical generator currently fails (Section 7.1). Section 9 makes the comparison on the five property axes.

The companion SharpeBench paper [Toca, 2026] supplies the scorer that consumes these trajectories: it argues that ranking trading agents on raw return over short windows measures luck, and its deterministic kernel deflates for search breadth [Bailey and de Prado, 2014], gates on per-run reliability [Yao et al., 2024], and zeroes entries that bypass risk controls. Every one of its guarantees is conditional on the trajectory being honest, which is the problem this paper takes.

SharpeArena excludes each of the three failures at the environment's own interface, then adds a detection layer behind the structural guarantee for defense in depth. Look-ahead is prevented by the shape of the data layer, which has no API that returns a future bar, and additionally policed by a deny-list guard at the harness boundary; this is an interface guarantee, and Section 10 states what it does not cover, including in-band prediction of the public deterministic generator. Determinism is achieved by restricting the scenario generator's arithmetic to operations that are exact and identical on every platform, and additionally pinned by committed golden hashes checked in continuous integration on the native and WebAssembly builds. Trajectory honesty is achieved by recording only decisions and recomputing everything else, and additionally exercised by a test that doctors a trajectory and asserts the replay diverges.

A fourth failure appears only once the agent is a separate process rather than a Python object, which is the arrangement a language-model agent forces. The transports that carry a decision over a wire have no way to report a wire fault through the interface they implement, so a timeout or a dropped connection arrives as an empty decision, and an empty decision against a stateful engine is a hold that persists the position. A wedged agent therefore completes its window and emits a return series that no scoring gate can distinguish from a deliberately conservative one, because every gate consumes returns. Section 4.4 converts a recorded fault into a typed failed cell instead, and Section 5 applies the same rule to the two other places the agentic layer can silently substitute a plausible default for a fact it does not have.

The interface follows the pattern that made Gym the substrate of a decade of reinforcement-learning research [Brockman et al., 2016, Towers et al., 2024]: a small, stable boundary that outlives any single

environment behind it. An agent is a program in any language that reads a JSON `Observation` and writes a JSON `Decision`. The contract is versioned, additive-only, schema-published and governed by a written deprecation policy, so a vendor can build against it once. Conform to the contract and you are scorable everywhere the SharpeBench kernel runs. Whether an ecosystem forms around this contract is an adoption question the paper does not answer; Section 10 records that no third party has yet submitted an agent.

Scope and contribution boundary. This paper claims the environment, its agentic layer and its protocol: the interface-level leak-freedom, the cross-runtime determinism, the replay verifiability, the wire contract and the fault typing on it, the strategy-generation and forward-execution paths, the seed-band generalization protocol, and the probe layer, together with the calibration experiments that exercise them. The scoring semantics, the deflation prior and the unit bug in the pre-0.5.0 kernel are the companion SharpeBench paper's contributions [Toca, 2026]; F1 appears here only as validation that the producer-scorer separation catches scorer bugs. Two absences are separate and both belong here rather than in a late caveat. No agent is trained in this paper: every experiment uses fixed reference policies, and a learning baseline through the shipped Gymnasium adapter is named future work in Section 10. And no agent field has been run: the agentic layer of Section 5 is claimed as architecture with verifiable guarantees, not as results, because no open-weight model has completed the predeclared field and no third party has submitted an agent (Section 10). The word agentic in the title describes what the environment drives and what it types as evidence, and it is not a claim that model performance has been measured.

Contributions.

1. A PIT RL environment for trading agents that is leak-free at its interface boundary, cross-runtime byte-deterministic in its Rust core, and replay-verifiable, with the enforcing code identified for each property.
2. A governed agent wire contract at version 1.0 with JSON Schemas, conformance fixtures and an additive-only evolution policy. It is the boundary an external program, a language model included, crosses to reach the engine, and it carries the fault typing of the next item.
3. Fault typing on the scored path. A transport fault is converted into a typed failed cell rather than the empty-orders hold the transports emit, because against a stateful engine that hold persists the position and produces a return series indistinguishable from a conservative agent's (Section 4.4). The same rule makes an unknown paper submission a representable state rather than a guessed one (Section 5.6), and records the decisions that were refused alongside the ones that executed (Section 5.5).
4. An evidence-admissibility discipline for the agentic layer: a closed falsifiability manifest binding each generated candidate to a hypothesis, a mechanism, invariants and quantitative kill conditions, recorded with a host-counted trial ordinal before validation (Section 5.4); and a strict silver-to-gold promotion path that refuses malformed production traces rather than skipping them (Section 6.5).
5. A procedural scenario generator with disjoint train, gap and held-out seed bands in the Procgen tradition [Cobbe et al., 2020], making the generalization gap a measured quantity for any policy submitted to it, though no learner is measured here (Section 10).
6. A task suite that includes a market-making environment shipping the analytical Avellaneda–Stoikov quoting policy [Avellaneda and Stoikov, 2008, Guéant et al., 2013] as a reference, so agents are scored on regret against a fixed analytical baseline rather than against each other.
7. A diagnostic layer that probes realism, manipulation payoff boundaries, adverse-selection mark-outs, population ecology and generator predictability. Two opt-in paths extend it: a concave-impact path that leaves the canonical linear golden path byte-identical, and a sealed-seed derivation that hides the held-out seeds behind a commit–reveal salt while the seed band structure keeps disjointness from the train band checkable by construction, without the salt.

8. The executed experiments and follow-ups of Sections 7 and 8, in which every number is produced by a committed script listed in Section A.

What the evidence shows. Every finding is an instrument calibration on canonical tape that fails the finite-panel-calibrated realism conjunction in 23 of 24 seeded panels, and a predeclared Calm calibration sweep finds no setting that survives its diagnostic, confirmation and volatility rule (Sections 7.1 and 7.1.1). Under the corrected kernel no reference policy is rank-eligible (Section 8.1), yet a common-random-number oracle witness attains eligibility on every tier and finds the every-seed probabilistic-Sharpe-ratio (PSR) gate last to open at every attained crossing (Section 8.1.1). The manipulation, adverse-selection, predictability and ecology probes each report against the environment as often as for it, and Sections 7 and 8 state the scope of each.

2 Design principles

Every decision in the environment traces to one of eight principles.

Leak-free at the interface. The environment owns the time cursor, and the data layer exposes no operation that returns a bar after it. An agent cannot peek through the interface because there is nothing to call, not because a rule forbids it. Detection layers exist, but they sit behind the structural guarantee, never in place of it. What an agent might infer in-band from the observations the interface does return, given a public deterministic generator, is a separate question, treated in Section 10.

Deterministic across runtimes. A published number must be byte-identical on any platform and any surface. The determinism-critical core is one Rust engine; its scenario transform uses only multiply, add, divide and max, because `ln` and `exp` differ across libm implementations. Golden hashes committed in the source pin the outputs, and the WebAssembly build asserts equality with the native engine rather than assuming it.

Recompute, do not trust. A trajectory is evidence only if a third party can recompute it. Runs record the agent's decisions and nothing derived; replay against the frozen scenario reproduces returns byte-for-byte, and a tampered trajectory reproduces something else. The counterfactual ledger of Section 5.5 extends the same rule from the decisions that were acted on to the ones that were not, because scoring only the acted decisions is a survivorship filter on the agent's own policy.

A fault is never a decision. When a failure has no representation, some plausible default absorbs it, and the default is indistinguishable from a real answer. A wire fault must not become a hold (Section 4.4), and a submission whose outcome is unknown must not become an accepted or an absent one (Section 5.6). The remedy in both cases is the same: make the failure state representable and typed, so that a downstream consumer must handle it rather than read past it. This principle is a property of the evidence rather than of the agent, which is why it sits beside determinism and replay instead of in an operations note.

The interface is stable under governance. The agent interface is a tiny JSON surface that evolves additively under a written governance policy, with schemas and conformance fixtures as the authoritative artifacts. An interface that moves under its implementers cannot be built against; stability under governance is what makes the contract worth conforming to.

Generalization is measured, not presumed. Scenarios are procedural functions of an integer seed, in the Procgen model [Cobbe et al., 2020], and the protocol fixes disjoint train, gap and held-out seed bands. The fixed-policy experiments use explicitly named subsets of those bands; a gap-band witness set is not described as the canonical held-out namespace. Overfitting becomes one number, the train-minus-test deflated Sharpe, instead of a suspicion.

The environment produces; the kernel judges. SharpeArena computes no rank. Scoring is delegated to the SharpeBench kernel [Toca, 2026], so the deflation, reliability and process gates are specified once, in one place, and the environment cannot quietly disagree with the scorer about what a good run is.

Distrust the simulator too. A synthetic market that pays manipulation without bound, that lets makers profit from informed flow, or that emits Gaussian tape will reward the wrong behavior. The probe layer treats the simulator as a subject: a finite-panel-calibrated realism diagnostic against selected stylized facts [Cont, 2001], payoff-boundary sweeps for manipulation, markout decomposition for adverse selection, and ecology runs that ask which strategies survive contact with each other. The probes report findings against the environment as often as for it: the realism diagnostic fails the canonical tape (Section 7.1) and the adverse-selection levels sit on the wrong side of this principle's own standard (Section 7.3).

The first three principles are properties of committed code, checkable by the listed tests. The empirical claims are properties of committed evidence artifacts and their producer scripts in the same source tree. Their versions, producers and hashes are stated once, in the provenance paragraph of Section A.

3 The environment

SharpeArena is a Rust workspace of three crates. The crate `sharpearena` holds everything determinism-critical: the stepping engine, the wire contract, the scenario generator, the limit-order-book matching engine, mandates and the execution-noise model, built on the published SharpeBench engine crates rather than a vendored copy. The crate `sharpearena-wasm` compiles the same kernel to WebAssembly and tests it against the native engine. The crate `sharpearena-py` binds the engine into a Python package that adds the ecosystem adapters: Gymnasium [Towers et al., 2024], PettingZoo [Terry et al., 2021], Minari [Farama Foundation, 2024], a `verifiers` training environment, and an MCP server. The adapters are thin by policy; anything that must be byte-identical lives in Rust.

3.1 Point-in-time observation, and the guard behind it

The engine owns the time cursor. On each step it emits a `MarketObservation` whose per-symbol snapshot carries a trailing window of closes up to the decision date, with optional fundamentals and news channels derived from that same trailing window. There is no API on the data layer that returns a bar after the cursor, so a policy cannot observe a future bar. The guarantee bounds what can be requested, not what can be inferred from the trailing window a request returns (see Section 10). A property test iterates every scenario tier and asserts that no surfaced window extends past the cleared bar and that each window is a suffix of the true history.

One layer out, at the agent-harness boundary, sits `LookaheadGuard`, a refusal layer for tools and policies that try to read what the environment will not give. It carries a deny-list of named operations, each mapped to the reason it is refused: reading the raw dataset (the answer key), slicing past the current index, peeking the next bar, or feeding the scenario seed to the policy as a feature, which would identify the episode and trivialize generalization. Substring tells catch novel operation names, and an index check bounds any point-in-time slice: reading an index strictly greater than the current bar raises. A research escape hatch exists as an explicit environment variable, so relaxing the guard is a recorded decision rather than an accident. The guard is defense in depth behind the structural guarantee: a harness that bypasses it still cannot make the environment leak.

3.2 Observation and action spaces

An implementer sees two JSON documents and nothing else. Table 1 gives the observation, Table 2 the decision; both schemas are draft 2020-12, both close the object with `additionalProperties: false`, and both are committed beside the conformance kit of Section 4.

The observation is flat. `date` is the clock: an ISO-8601 date naming the decision point, and the bound every other field respects. `cash` is the agent's own cash. `symbols` carries one snapshot per tradable instrument, and `portfolio` the agent's own holdings, never a peer's pending state. A snapshot's `close_history` is the trailing window: the last L closes up to and including `date`, oldest first, where L is the scenario's disclosure setting. The default is 20 closes with no optional channels; the shipped presets are a data-poor tier at 3 closes and a data-rich tier at 50 closes plus fundamentals

Table 1: The `MarketObservation` wire object at contract v1.0. Optional fields default to empty and are absent from the schema's `required` list by construction, which is what makes an additive field backwards-compatible.

Field	Type	Presence	Meaning
Object `MarketObservation`			
`date`	string (ISO-8601)	required	decision point; the point-in-time cutoff
`cash`	number	required	cash available to this agent
`symbols`	array of `SymbolSnapshot`	required	one snapshot per tradable instrument
`portfolio`	array of `PositionState`	required	this agent's holdings; empty at a cold start
Object `SymbolSnapshot`			
`symbol`	string	required	instrument identifier
`close_history`	array of number	required	trailing closes through `date`, oldest first
`fundamentals`	map string to number	optional	derived fields; empty by default
`news`	array of string	optional	derived headlines; empty by default
Object `PositionState`			
`symbol`	string	required	instrument identifier
`shares`	number	required	signed holding; negative is a short
`avg_price`	number	required	displayed average entry price

and news. Both optional channels are derived from the surfaced window itself and therefore inherit its cutoff: `fundamentals` is a named map computed from those closes (trailing return, window high, window low) and `news` is one deterministic headline per symbol whose wording is a function of the window's trailing return. The v1.0 observation carries no mandate field. The mandate is a scenario-level object (Section 3.7) rendered into the episode prompt by the Python task layer, so the fixed reference policies of Section 8 act mandate-blind, which is what the failure taxonomy of Section 8.4 counts.

The action space is a signed target-weight vector over the observed universe. A `Decision` is a list of per-instrument orders plus an optional free-text `reasoning` string that the trajectory records. An empty order list is a hold. Within an order, `action` is a label drawn from a closed four-variant enum (`buy`, `sell`, `hold`, `close`) and carries no sizing; all sizing is in `target_weight`, which is a signed target portfolio weight for that symbol. The sign is the shorting semantics: a negative `target_weight` is a short position, and the `signed-weight-short` conformance fixture exercises exactly that case, pairing `action: sell` with `target_weight: -0.3` while a sibling order holds at 0.0 and a third buys at 0.5. Adding a fifth action variant is classified as breaking (Section 4) precisely because a consumer that matches the enum exhaustively would misparse it.

Four bounds are asserted by the conformance kit rather than by the schema's type system: every `action` lies in the enum, every `target_weight` is finite, every order `symbol` is drawn from the universe the observation actually offered, and every `confidence` lies in $[0, 1]$. The schema places no numeric bound on `target_weight` itself; the gross-exposure ceiling an episode is graded against is a property of its sampled mandate (Section 3.7). A legacy decision that omits `confidence`, `rationale` and `reasoning` still parses and is still well-formed, and the `legacy-decision-shape` fixture asserts it on every commit. Because both schemas close the object, a field the engine emits but the schema omits is an interoperability break rather than a documentation gap, so the conformance kit compares the serialized keys of fully-populated wire values against the schema's declared properties in both directions, for the observation and the decision alike.

3.3 Determinism across runtimes

A scenario is a pure function of one 64-bit seed. The generator's price transform uses only multiply, add, divide and max; it never calls `ln` or `exp`, which differ across libm implementations, so a generated panel is byte-identical on any platform and any runtime. The source commits FNV-1a fingerprints of canonical generated scenarios and of a scripted order sequence's fill tape. Tests recompute the hashes on every commit, and WebAssembly parity tests require native-identical returns, traces and scenarios. The generator is deliberately public and deterministic; that makes the hashes checkable

Table 2: The `Decision` wire object at contract v1.0. Optional fields carry the stated default, so a decision written against an earlier revision of the surface still parses.

Field	Type	Presence	Meaning
Object `Decision`			
`orders`	array of `Order`	required	per-instrument instructions; empty is a hold
`reasoning`	string	optional (`""`)	rationale captured into the trajectory
`cost`	`DecisionCost`	optional (absent)	self-reported spend for this decision
Object `Order`			
`symbol`	string	required	identifier; must be an observed symbol
`action`	enum	required	`buy`, `sell`, `hold` or `close`; label only
`target_weight`	number	required	signed target weight; negative is a short
`confidence`	number	optional (0.5)	stated conviction in $[0, 1]$
`rationale`	string	optional (`""`)	per-order rationale
Object `DecisionCost`			
`cost_usd`	number	optional (0)	dollar cost of the compute spent
`tokens_in`	integer	optional (0)	prompt tokens consumed
`tokens_out`	integer	optional (0)	completion tokens produced
`reasoning_tokens`	integer	optional (0)	reasoning tokens, not re-added to the total

and future bars computable from a recovered seed. Section 7.4 measures the boundary: a causal AR adversary finds no undisclosed stressed-tier edge, but a public bounded band of 2^{16} seeds is exhaustively inverted from one observed bar in about one second. Held-out evaluation therefore requires high-entropy unpublished seeds, not merely disjoint public bands.

3.4 Recompute-from-decisions tamper evidence

A rollout trace is a versioned JSONL artifact that records the agent's decisions and rejects anything leaky: the writer refuses values that would embed future data in the record. Everything else, returns included, is recomputed at verification time by replaying the decisions against the frozen scenario, and because the engine is deterministic the recompute is byte-identical. The npm package's test suite exercises the adversarial case directly: it captures a trajectory, doctors every order's target weight, replays both, and asserts the tampered artifact cannot reproduce the honest returns. An agent cannot lie about its performance to anyone willing to run the replay.

3.5 Procedural scenarios and disjoint seed bands

Scenario families follow Procgen's integer-seed-interval model [Cobbe et al., 2020]: Calm, Hard and Extreme volatility-and-jump tiers, a cointegrated-pairs family with a mean-reverting spread, and a regime-shift family that hands a trend regime over to a whipsaw. The Rust core exposes `train_test_split`, which carves a disjoint test family (two separated integer intervals, with the disjointness asserted in tests) from a bounded train interval and refuses an unbounded one, and `cross_regime_split`, which holds the seed band fixed while swapping the regime, a different and generally harder probe than a within-tier split. The evaluation protocol fixes the canonical bands: train seeds $[0, 256)$, an unsampled gap of 10,000 seeds, and a held-out test band $[10256, 10512)$. The fixed-policy F3 and witness producers instead document their own 16-seed subsets: $[0, 16)$ and $[10016, 10032)$; the latter is a separated gap-band evaluation subset, not the canonical held-out namespace.

The Python layer additionally reserves an absolute evaluation namespace starting at seed 10^6 and commits a named held-out regression set inside it, with a disjointness guard asserting the set never touches the train band. All of these bands are public and therefore enumerable; for an adversarial evaluation the recommended mode is the sealed derivation of Section 7.4.1, which keeps the band structure checkable while hiding the concrete seeds behind a salt.

Overfitting is then a measurement. `generalization_gap` scores the same policy on disjoint named bands and reports the differenced deflated Sharpe, and `cross_regime_transfer` reports the in-

distribution minus zero-shot out-of-distribution gap over identical seeds, which is zero by construction when the regimes coincide.

3.6 The task suite and the closed-form reference policy

Beyond the canonical position-trading environment, the suite ships a simplex-allocation portfolio task, a VWAP/TWAP implementation-shortfall execution task, and two market models: an endogenous shared-book market where aggregate agent flow moves the cleared price through a Kyle permanent-impact term [Kyle, 1985] and an Almgren–Chriss temporary-impact term [Almgren and Chriss, 2001], and a deterministic limit-order-book market with integer ticks, price-time priority, a single-price call-auction uncross and a read-only walk-the-book sweep-cost query.

Both market models clear once per step at a single price, so the trading mechanism is closer to a frequent batch auction [Budish et al., 2015] than to a continuous-time limit order book. The competition margins that continuous markets create, latency races, queue-position value, sniping of stale quotes, do not exist in the environment, and no maker-taker fees or rebates are modeled: every Sharpe, regret and markout in Sections 7 and 8 is gross of fees. The restriction is deliberate, it keeps clearing deterministic and byte-reproducible, and it makes the environment a batch-auction laboratory rather than a latency one, but it is a scope restriction and results should be read inside it.

The market-making task is the suite's calibration instrument. It implements the Avellaneda–Stoikov model [Avellaneda and Stoikov, 2008] and ships the model's closed-form quoting policy beside the environment: reservation price $r = S - q\gamma\sigma^2\tau$ around the mid price S and half-spread $\delta^* = \gamma\sigma^2\tau/2 + \gamma^{-1}\ln(1+\gamma/\kappa)$, skewed by inventory. This closed form is the source model's asymptotic approximation, not an exact optimum: the exact treatment of the same problem is given by Guéant et al. [2013], and no proof is offered that the policy is optimal for the implemented reward, which adds an inventory cap, a running inventory penalty and a terminal liquidation charge the source model lacks. We therefore call it the closed-form reference policy. Both the environment and the policy are pure functions of the same frozen parameter set, so the regret metric `mm_regret` runs the candidate and the reference on identical seeds and reports the mean reward gap; the reference scores zero against itself by construction, which fixes the metric's zero point without claiming the zero point is unbeatable. The model also assumes fills that do not select against the maker, which sits in deliberate tension with the adverse-selection probe of Section 7.3. Gym-style market-making environments built on the same model family exist [Jerome et al., 2023]; what this task adds is the shipped reference policy and the regret-not-PnL scoring convention inside the verifiable-producer stack.

3.7 Mandates and deferred claims

Each scenario can sample a trading mandate, and the objective is conditioned on it. A mandate pairs one of five structural styles, long-only, market-neutral, momentum, unconstrained and pairs-convergence, with up to three optional constraints layered on top of that style: a realized-drawdown cap, a per-bar gross-exposure cap on $\sum_i |w_i|$, and a benchmark symbol to beat. Unconstrained is the permissive control, and the two styles that need a short leg, market-neutral and pairs-convergence, are excluded when the environment disallows shorting, so a sampled mandate is never unsatisfiable by construction: `mandate_breach` detects structural violations from the realized weights and returns, and the training rubric penalizes wrong-objective behavior rather than merely not rewarding it. A deterministic failure taxonomy classifies every episode into a single worst-case-first disposition, cascade-wiped before bankrupt before stopped-out before mandate breach before clean, and a suite-level rollup tallies the distribution with a schema that always carries every mode. The taxonomy ranges over completed episodes and reads their realized weights and returns, so a cell that failed at the wire (Section 4.4) is not a sixth disposition. It is the absence of an episode to classify, it carries a typed fault instead of a disposition, and it is counted apart from the rollup that Section 8.4 reports.

Questions that resolve after the episode get the same structural treatment as look-ahead. A `DeferredDesk` is the only object the agent touches: its entire state is a list of claims and one integer clock advanced by the harness, it holds no dataset and no environment reference, and it

computes no score, so there is no code path from a claim to its resolving datum. Resolution is a free function over claims and outcomes that refuses any outcome available at or before the claim's commit bar and any outcome predating the stated horizon. The agent cannot see the answer at commit time because the object it holds cannot produce one.

3.8 The probe layer: distrusting the simulator

Realism. A stylized-facts battery computes excess kurtosis, absolute-return autocorrelation, gain/loss skew and aggregational Gaussianity in the sense of the classical catalogue [Cont, 2001], plus a timescale-asymmetry statistic in the sense of Zumbach [2009] and an exploratory Fano exceedance-count ratio (the variance-to-mean ratio of the number of large-return exceedances per equal-length block, which is 1 for an IID Poisson-like count and above 1 when exceedances bunch). `certify_realism` gates kurtosis, an ACF value above the fixed 95th-percentile IID null for its exact short-panel estimator, and movement of absolute excess kurtosis toward zero on aggregation. Fano is reported but ungated: it is normalized by its exact conditional-IID finite-panel expectation, yet 12 count windows remain too noisy for a certificate. Certification is reported beside the results rather than required for the leaderboard, and the canonical configuration fails it on 23 of 24 seeded panels (Section 7.1). The intended endgame is a generator revision that passes on disjoint calibration seeds or leaderboard claims conditioned on certification; until then the gate keeps the failure visible.

Manipulation. A crude push-and-unwind schedule, the classic trade-based manipulation of Allen and Gale [1992], is scored against its own zero-impact reference: the live and reference runs share the seed and follower population, and differ only in impact. A boundary sweep varies one parameter and a size-response sweep asks whether payoff is bounded. Linear permanent impact makes the baseline a model-consistency check under the assumptions of Huberman and Stanzl [2004], Gatheral [2010]. On separate entry points, nonlinear permanent impact is $\lambda \operatorname{sign}(Q/V)|Q/V|^{\beta}$, with dimensionless flow Q/V and a dimensionless coefficient λ for each declared β; $\beta = 1$ uses the exact original linear arithmetic. This avoids overloading the market-making model's κ (order-book-depth parameter). Section 7.2.1 evaluates the symmetric schedule, and Section 7.2.2 reports a 135-cell exploratory asymmetric search with selection-adjusted inference.

Adverse selection. Informed meta-order flow is routed against a maker roster, with fills struck at the pre-move mid so the maker's markout carries the informed trader's edge. The mechanism the probe operationalizes is the classical one [Glosten and Milgrom, 1985], with one deliberate simplification: the price path is exogenous and quotes do not widen in equilibrium response to toxicity, so the probe measures markout accounting against drift, not equilibrium adverse selection. The design is a paired control: the informed leg sides child orders on the alpha, the uninformed leg sides the identical flow on a coin while holding the price path fixed, and the comparison reports mean markout per filled unit at each horizon. If the informed leg does not mark out worse, the module's own numbers are declared noise.

Ecology. A replicator dynamic in the market-ecology tradition [Farmer, 2002, Lux and Marchesi, 1999] runs a population of strategies over generations: fitness earned in a shared-book market where a species that gains share becomes a larger fraction of the flow everyone else fills against, seat allocation from population shares, extinction below a threshold, an innovation operator that breeds parameter-perturbed variants, and shock schedules that rotate regimes or scale impact to simulate liquidity droughts. The control schedule holds the environment steady, so any claim about a shock has a baseline to beat.

4 The agent contract and its governance

An agent is an external program in any language that reads a `MarketObservation` as JSON and replies with a `Decision` as JSON. The harness sends an observation, the agent returns a decision, repeat. Everything else in this paper exists to make that loop trustworthy.

4.1 Authoritative artifacts

The contract has four authoritative artifacts, all committed. `CONTRACT_VERSION`, currently `1.0`, is the single source of truth for the wire version and lives in the contract module of the core crate. Two JSON Schemas, draft 2020-12, define `MarketObservation` and `Decision` for non-Rust implementers; Section 3.2 enumerates their fields. A conformance kit of fixture documents, covering the normal multi-symbol case, a single symbol, an empty-portfolio start, signed-weight shorting and a legacy decision shape, is exercised by a committed conformance test. And a governance document states the evolution rules in writing. The schema closes the JSON shape but cannot know the symbols in the current observation, so one canonical runtime parser additionally enforces episode semantics. The MCP, training and local-agent surfaces all call it; a semantic fault returns without advancing the environment.

4.2 Additive-only evolution

The contract evolves additively. The only backwards-compatible change is a new field that is optional with a default, so an agent written against an older version keeps parsing newer observations and the harness keeps parsing older decisions. The optional fields are absent from the schemas' `required` lists by construction. Removing, renaming or retyping a field, making an optional field required, and removing or renaming an action variant are all defined as breaking and forbidden on the v1 surface. A breaking change never mutates the v1 types in place; it ships a parallel namespace with its own schemas and its own major version, and the two run side by side through a published deprecation window. Adding a new action variant is itself classified as breaking, because consumers that exhaustively match would misparse it, so it also goes through the parallel namespace rather than an additive bump.

4.3 Version semantics

`CONTRACT_VERSION` versions the wire shape only and is deliberately decoupled from the crate and package versions, which move on their own release cadence. A major bump is a breaking change shipped as a parallel namespace. A minor bump is reserved for a batch of additive fields significant enough to advertise; a single additive field needs no bump at all, because existing agents are unaffected by definition. A package release never, on its own, bumps the contract version.

4.4 A transport fault is a failed cell, not a hold

An external agent speaks over a wire, and a wire fails. The v1.0 `Decision` carries no error variant, and the external transports cannot signal one through the `Agent` trait they implement, so a timeout, a dropped connection or an unparseable reply degrades to an empty order list while the fault itself is recorded into a rolling `TransportHealth`. Section 3.2 fixes what an empty order list means: it is a hold, and against a stateful engine a hold is not inaction. The position persists. An agent whose wire wedges on the first bar therefore rides its opening position to the end of the window, the run completes without raising, and the return series it emits is the return series of a deliberately conservative agent. No downstream gate separates the two, because the deflation, reliability and process legs all consume returns. A committed characterization test states the failure rather than describing it: a wedged agent run through the plain engine call yields a scoreable `Run`.

The health record was the only mitigation, and a record mitigates only if something reads it. Nothing did. This crate re-exported neither `TransportDiagnostics` nor `TransportHealth`, so no consumer could name the type, let alone read it, and a search of the repository found no reader anywhere. Two changes close that. The diagnostics types are re-exported from the crate root, and `run_backtest_checked` replaces the plain engine call for anything that speaks over a wire: it runs the cell, reads the health afterwards, and returns a `CellOutcome` that is either `Scored(Run)` or `Failed(TransportFault)`. The typed failure carries the retryable transport-fault count, the agent-protocol fault count, whether the per-endpoint circuit breaker tripped, and the last decision-level

error. Its accessor returns `None` on a failed cell rather than a default-valued `Run`, so a caller cannot score a failure by accident. Four further tests cover the guarded path: a wedged agent is a failed cell, a clean transport is scored, an unparseable reply is also a failed cell, and an undegraded health record reads as no fault at all.

The fix belongs with the determinism and replay guarantees rather than with operations, because it is the same kind of claim. Leak-freedom bounds what the agent could see and replay bounds what the trajectory could be; neither says anything about a return series the harness synthesized on the agent's behalf. Masking is the one failure mode that produces evidence a verifier cannot reject, since the doctored quantity originates inside the honest producer and replays byte-for-byte. A failure is evidence, and never a hold.

4.5 Why the contract can be trusted without trusting the host

The governance promise is social; the properties of Sections 3.1, 3.3 and 3.4 are technical, and together they remove the host from the trust equation for everything the byte-identity guarantee covers. That coverage has a boundary. The Rust core, its WebAssembly build and the scenario bytes returned through the Python bindings are pinned to the byte by committed FNV-1a goldens, which the native and Python suites assert in full and the WebAssembly suite asserts for the canonical and clustered scenarios, as are the engine outputs behind F1 through F3. The Python evidence layer behind the remaining probes is deterministic in its seeds but sits outside the golden-hash path (Section 10). Within the covered surface, a leaderboard entry can be recomputed from the frozen seed and submitted decisions on any tested platform. The scoring rules are defined independently by the SharpeBench kernel's deflated Sharpe, reliability and process gates [Toca, 2026, Bailey and de Prado, 2014].

5 The agentic layer

The contract of Section 4 defines a loop. This section describes what the environment puts around that loop when the program on the other side of it is a language model: a batched local-model field, a strategy-generation path that counts its own search, a ledger of the decisions that were not acted on, and a forward paper-execution arm. Everything here is architecture and its guarantees. No agent result is reported from any of it, and Section 10 states that absence as a limitation rather than leaving it to be inferred.

5.1 Local agents and the scorer boundary

The local-model path keeps environment execution and field judgment separate. SharpeArena owns the point-in-time observation loop, canonical decision parser, execution, process trace and append-only cell journal. A compiler validates that a journal contains the complete predeclared Cartesian field, with one consistent completion per cell and matching return hashes, then emits an ordinary SharpeBench submission artifact for each dataset. SharpeBench owns the field-level deflation, bootstrap, process and reliability judgments. The implementation dependency remains directed: SharpeArena uses the small published SharpeBench protocol, simulator and kernel crates, while SharpeBench does not import the full environment package. The products are therefore operationally interdependent without a package cycle.

Model output crosses one canonical boundary. The published `Decision` schema constrains syntax, and a host-side parser separately enforces episode semantics: symbols must have been observed, each symbol may occur once, weights must be finite and within the configured bound, and action labels must agree with their signed targets. The MCP, training and local-model paths call that same parser. A malformed response, timeout or transport failure is recorded as a typed fault and, through the gate of Section 4.4, becomes a failed cell rather than a hold or a flat return series. The field record binds the model artifact and server version, sampling and inference seed, cadence, thinking setting, prompt/scaffold digest, raw response hash, token counts, latency and exact process events.

The shipped scheduler batches the native vector environment, assigns stable cell ordinals over model, scenario, seed and repetition, supports deterministic sharding, and resumes through append-only journals. No local open-weight model result is reported in this paper.

5.2 Where the byte-identity guarantee stops

The producer/scorer boundary of Section 6.4 has two parties. An agentic field has three, because the model provider is neither the environment nor the kernel, and this is the exact point at which the paper's central guarantee ends. Environment replay stays deterministic: the same seed produces the same scenario bytes, and the same decisions replay to the same returns on any tested platform. Model inference does not. Continuous batching, server version, sampling configuration and hardware all move the sampled token sequence, so the paper treats inference as a measured source of agent variance rather than describing it as byte-identical. What the field record can pin is everything around the sampling step: which checkpoint, which server, which seed, which prompt digest, which raw response hash. What it cannot pin is that the same checkpoint and seed will re-emit that response. Environment bytes and replay remain fixed while agent outputs vary, and the evidence schema keeps the two apart rather than averaging over the distinction.

Local inference is a trusted loopback service and sits inside the trusted computing base. Containment for an executable third-party entrant is a different problem and is not this artifact's; Section 10 states whose it is and what has been verified about it, which is less than the word sandbox usually implies.

5.3 Observed strategy search

The strategy-generation path measures the search performed inside the harness. A model emits a closed JSON language over bounded price, moving-average, momentum, RSI and volatility conditions; it cannot emit Python, shell commands or arbitrary tool calls. The host records the raw generation response and counts every candidate before validation or deduplication. Accepted candidates are compared on an explicit validation interval under a deterministic median-score rule with a fixed tie-break; only the selected candidate touches the disjoint test interval. That observed raw count supplies the deflation trial count for this search, which is the count Section 6.4 asks an entrant to declare honestly and here does not have to take on trust. It does not reveal candidates generated before entry, so it narrows rather than closes the declared-trials problem.

5.4 Binding a candidate to a falsifiable claim

A candidate that arrives with a rule and a backtest number invites the prose that follows selection. The hypothesis, the mechanism and the conditions under which the edge would be abandoned all get written after the winner is known, and are shaped by knowing it. The `EdgeManifest` forecloses that ordering by making the falsifiability terms part of the candidate, under a closed schema, recorded before validation and before deduplication.

A manifest binds a candidate to a hypothesis and a mechanism, each required to be at least eight characters so that an empty string is not a manifest, together with the regimes and instruments the claim is made for, at least one invariant, at least one kill condition, and a verification plan. Every condition applies a comparator from a closed set to a threshold that carries an explicit unit from a ten-member enumeration, so a bare number is a parse error and a string such as `"2%"` is rejected with the instruction to write the number and put the unit in the unit field. A missing required field invalidates the candidate; nothing is defaulted, and an unknown field is refused alongside a missing one. An empty invariant list is refused on the stated ground that it asserts nothing and can therefore never fail, and one condition identifier may not appear in both the invariant list and the kill list. The verification plan refuses a confirmation split equal to the selection split, and requires at least two observations before it will conclude anything.

The ledger assigns a candidate its trial ordinal before validation and before deduplication, so the count it publishes is the count of candidates the model emitted rather than the count that survived. An

invalid candidate is recorded with the reason it was invalid, a repeat is recorded with the ordinal it duplicates, and neither is dropped. The summary labels the count with the stage it was taken at, so a downstream deflation prior consumes the pre-filter number and not a filtered one. A ledger whose trial ordinals do not match its own row count fails to load rather than loading partially.

Kill conditions are evaluated only outside the selection sample: the monitor refuses any sample not named as test, forward or holdout. It returns one of four verdicts under a precedence fixed in code. A fired kill condition retires the edge; failing that, an unresolved check makes the whole report indeterminate; failing that, a violated invariant reports a violation; otherwise the edge is healthy. A metric that is absent, that arrives in a unit other than its threshold's, or that is not a finite number resolves to unresolved rather than to a pass, so a measurement that was never taken cannot be read as an edge that survived.

5.5 The counterfactual ledger

A journal of executed orders records what the risk guard let through and says nothing about what it stopped, so the distance between what the agent proposed and what the venue received is not a measurable quantity. Scoring only the acted decisions is a survivorship filter, and the thing it filters is the agent's own policy. The counterfactual ledger records every decision, acted on or not, with one ghost order per intended order carrying the intended quantity, the quantity that actually executed, a reference price, and one disposition from a closed five-member set: executed, risk refused, below minimum notional, not submitted, or resized.

The record refuses to contradict itself. A disposition of executed whose intended and executed quantities differ raises; a disposition other than executed whose quantities agree raises with the complaint that it claims a difference that is not there; an executed quantity above the intended one raises; and the flag saying the decision was acted on must agree with whether any quantity reached the broker. Over a window the ledger reports a selection gap: how many decisions were acted on, intended against executed notional, intended against executed profit and loss under supplied settlement prices, and the difference between them, with a disposition histogram that always carries all five keys so a zero is printed rather than omitted. A positive foregone figure says the execution path cost money and a negative one says it avoided a loss. Neither is a claim about the market; both are claims about the gap the harness introduced.

In the deterministic arm the ledger is free, because every ghost order is recomputable from the frozen scenario. It defaults to memory there and adds no artifact to the replay path.

5.6 Forward paper execution and the unknown submission

Forward paper trading is a different evidence class. Read-only market data enters a trusted supervisor; the model sees observations but never credentials; a fixed risk guard preflights the order batch against a shadow account before an in-memory broker or an adapter pinned to Alpaca's paper endpoint can receive it, and that adapter refuses any other origin and follows no redirect. The preflight stops at the first refusal, because a remote paper broker does not update the caller's account between requests and the verdicts after that point would be computed against a state the broker never reached; the orders it never assessed are recorded as not submitted, which Section 5.5 keeps distinct from refused. The journal records the market snapshot, decision, proposed orders, risk outcome and paper-broker response, and a separate private preimage supports the forward commitment. There is no real-capital endpoint and no override. Provider time, provider data and simulated fills make this evidence non-replayable by construction, and every record the forward journal writes is stamped as non-deterministic with its replay guarantee named as none, so a forward result may extend the forward record but may not inherit the byte-identity claim of the historical environment.

The state that matters in a forward arm is the one a backtest never has. A submission whose response never arrives is neither accepted nor absent, and the harness cannot know which without asking the broker. The order lifecycle therefore makes `submission_unknown` a representable state rather than a comment. It is reachable only from `submitted`, and it leaves only to `reconciled_accepted`

or `reconciled_absent`. A server-side error or a transport failure on submit puts the lifecycle there, while a broker that answers with a client-side rejection is a rejection and not an unknown. Acknowledgment latency stays unset until a real acknowledgment arrives, so an order that reaches its terminal state through reconciliation does not acquire a fabricated one.

Reconciliation asks the broker by a client order identifier derived from the agent identity, the model digest, the parsed decision, the market snapshot and the order index, so the same decision addresses the same order on every attempt and a retry cannot create a second order by accident. A replacement is authorized only after the broker confirms absence, and only once: the lifecycle refuses a second. A query that cannot be answered raises rather than resolving, so a network failure is never recorded as a confirmed absence, and the lifecycle stays in the unknown state where no replacement is authorized. State is persisted before the submit call, again on the transition to unknown, and again after every reconciliation, each write going to a temporary file that is then atomically renamed, so an order left unresolved by a crash is still unresolved after the restart rather than quietly absent. Account state is reconciled against the broker rather than read from the plan file and left stale. Every transition and every raw broker acknowledgment hash reaches the forward evidence journal, which stays distinct from the deterministic backtest evidence.

Two consequences of this design are limitations rather than features, and Section 10 states both: partial fills accumulate by last write rather than by summing deltas, and a broker that cannot answer a reconciliation query halts the session instead of retrying.

6 Benchmarking protocol

This section is written for someone who wants to evaluate an agent in SharpeArena rather than for someone reading our results. It fixes the seed bands, the seed counts, the seed-custody mode, the scoring path and the verification step. The rules below are the ones our own producers follow, stated prescriptively; where a producer departs from them, we say so and say why.

6.1 Seed bands: what to train on and what to evaluate on

Train on the bounded band $[0, 256)$. Evaluate on the held-out band $[10256, 10512)$. The unsampled gap of 10,000 seeds between them is part of the protocol, not a formatting artifact: adjacent integer seeds are cheap to enumerate from a published train interval, and the gap makes an accidental overlap visible as an out-of-band seed rather than as a plausible one. `train_test_split` carves that structure and refuses an unbounded train interval; the disjointness is asserted in the crate's tests rather than left to the caller.

Two bands that appear in this paper are *not* the canonical held-out namespace, and results reported on them should not be described as held-out results. The fixed-policy F3 producer uses a 16-seed train subset $[0, 16)$ and a 16-seed gap-band evaluation subset $[10016, 10032)$; the rank-eligibility witness of Section 8.1.1 uses the same gap-band subset and, separately, the F1 table band. Both subsets are separated from the train band, and neither is $[10256, 10512)$. We use them because the fixed reference policies have nothing to overfit, so the smaller subsets cost nothing in interpretation and buy tractable runtimes for a probe repeated under five noise paths per cell. A submitted agent has parameters and therefore does not get that exemption: report it on the full canonical band.

Report the cross-regime transfer matrix alongside the within-tier gap. A within-tier gap is structurally blind to the quantity Section 8.3 measures, namely how much score a regime switch moves for a policy selected on one tier, and both metrics run from the same committed scripts. The Python surface additionally reserves an absolute evaluation namespace starting at seed 10^6 with a committed held-out regression set inside it and a disjointness guard against the train band; use that namespace for regression testing a harness, not for reporting agent scores.

6.2 Seed counts

Run all 256 seeds of the held-out band on each tier you report. The reason is resolution, and this paper measures it directly. At 16 seeds, a per-seed pass rate near one half carries a nominal 95% half-width of about 0.24, and a difference between two such rates about 0.35 (Section 8.1); a bounded, heavily deflated score bootstrapped over 16 seeds produces intervals that span most of $[0, 1]$ (Section 8.3); and on the market-making task, 16 paired episodes detect a mean regret of about 12 reward units at 80% power and a two-sided 5% level, roughly a third of the regret a badly miscalibrated quoter incurs (Section 8.2). Sixteen seeds resolve the shape of a curve and the sign of a collapse. They do not resolve a ranking between two nearby agents, and a leaderboard built on them would be reporting seed luck.

State the dispersion convention beside every number. We report three: seed-resampled percentile bootstrap intervals for deflated-Sharpe quantities, t-based intervals over committed per-episode vectors for paired quantities, and plain counts where the quantity is a count. Whichever convention you use, serialize the per-seed or per-episode vector behind the interval, as the producers under `paper/src/` do, so a reader can recompute it. Any claim narrower than the reported interval is outside the design's reach and should not be made.

6.3 Seed custody, and what is forbidden

Every band named above is public and therefore enumerable. Section 7.4 measures what that costs: a scan of a public 2^{16} band recovers the true seed from a single observed bar in about a second, and a recovered seed makes every future bar of the episode exactly computable. For any evaluation whose result is meant to survive an adversarial reading, use the sealed derivation of Section 7.4.1: publish the salt hash before entries arrive, withhold the salt during scoring, and reveal it afterwards so anyone can recompute the named seeds and replay the run. The band structure stays checkable without the salt, so disjointness from the train band remains verifiable by construction.

Any public bounded-band evaluation should be treated as compromised until it documents sealed seed custody, a pre-entry commitment, a non-reuse policy and a reveal deadline. This is a statement about the protocol, not an accusation about any particular run: the scan is cheap enough that assuming it was not performed is unreasonable.

Three prohibitions bind a submitted agent, and each has an enforcing layer rather than a promise. Reaching past the cursor is forbidden: reading the raw dataset, slicing past the current index and peeking the next bar are refused by `LookaheadGuard` at the harness boundary and are structurally unavailable at the environment's own surface (Section 3.1). Feeding the scenario seed to the policy as a feature is forbidden, because it identifies the episode and makes the generalization protocol meaningless. Enumerating or reconstructing the evaluation seeds, by band scan or otherwise, is forbidden, and sealed derivation is what makes that prohibition enforceable rather than honour-based. Relaxing the guard is possible for research use through an explicit environment variable, which records the decision; a run that sets it is not a protocol-conformant evaluation.

6.4 Scoring and verification

The environment contains no scorer, and an entrant does not supply one. Scores are computed by the SharpeBench kernel: deflated Sharpe with the declared trial count [Bailey and de Prado, 2014], a per-run reliability gate, and the process gates that zero entries bypassing risk controls [Toca, 2026]. Rank eligibility is the conjunction of those legs, not any one of them, and Section 8.1 shows why: deflation alone crowns drift on the calm tier, and the reliability gate alone misses the search breadth deflation discounts. Declare the trial count honestly. Section 8.3 contains a worked instance of what happens otherwise: the same policy on the same sixteen seeds scores 0.1231 or 0.1114 depending only on how many trials the deflation prior is told to consume.

Submit the trajectory, not the returns. A conformant trajectory is a versioned JSONL artifact recording the agent's decisions; returns, NAV and every derived quantity are recomputed at verification time

by replaying those decisions against the frozen scenario (Section 3.4). To verify a submission, regenerate the scenario from the declared seed and pass it with the trajectory to the replay verifier (`replay_submission` in the Rust core, `replayRun` in the npm package), then check that the recomputed run is byte-identical to the reported one. Because the engine is deterministic across the covered surface, a mismatch is evidence of tampering or of a misdeclared scenario, and the npm package's test suite exercises exactly that case by doctoring every order's target weight and asserting the replay diverges. A reviewer who runs the verifier does not have to trust the entrant, the host or us.

6.5 Growing the case set: strict silver-to-gold trace promotion

An evaluation suite written by the people who wrote the environment tests what those people already thought of. The alternative is to grow it from traces the system actually produced, and the hazard there is subtler than a bad case: a pipeline that skips whatever it cannot parse silently selects for the traces that happened to be easy to read, and the resulting suite is biased by the parser rather than by the operator. The promotion path refuses instead of skipping.

A malformed or incomplete trace raises rather than being passed over. A blank line, a line that is not JSON, a record that is not an object, a step record appearing after the metadata record, a missing observation, a missing decision, a missing reward (a missing reward is not zero), a non-finite reward, a step ordinal out of sequence, a declared step count that disagrees with the steps present, or a trace shorter than two steps all abort the load. Eight metadata fields are required and none of them is defaulted.

A trace is identified by a fingerprint over six components: environment, model, scaffold, contract, data and process. The data component digests the dataset hash together with the sorted scenario seeds. The process component digests the sequence of process events, recording per step the ordinal, the order count, the sorted action labels, the sorted environment events and whether the episode terminated. It is derived from actions rather than rewards, so two runs that arrived at the same outcome by different routes do not share a fingerprint.

Deterministic checks run before anything else, and four of the five block: rewards finite, no look-ahead key present in the observation, a structured decision, and seeds declared in the metadata. The fifth, that the reward series varies at all, only warns, because a flat series is a suspicion rather than a defect. Only a failing check produces a silver candidate, and what is frozen is not the transcript. The scenario is minimized by a greedy two-sided shrink to the smallest window on which the check still fails, floored at two steps, with the per-step auxiliary payload stripped to four keys; what is stored beside it is the check that triggered the promotion and the hash of the source trace. Silver rows are immutable: re-appending an identical row is a no-op, and appending different content under the same identifier raises.

Promotion from silver to gold requires a recorded operator decision that names the operator, names the candidate it promotes and carries a rationale of at least twenty characters. A gold case is a minimal scenario plus the invariant it is expected to satisfy, and it is evaluated with sockets disabled, so a case that reaches for the network fails rather than succeeding against a live service. This is the same admissibility discipline as seed custody, applied to the case set instead of to the seeds, and it is what keeps the forward and generated-strategy evidence of Section 5 from being pooled with the deterministic tables of Sections 7 and 8 on the strength of looking similar.

6.6 Compute cost

The protocol above is cheap, and we measure that rather than assert it. Table 3 reports throughput for the three shipped surfaces on the canonical 4×120 panel, measured on one AMD Zen 3 desktop under Windows 11 by the single command in Section A, which times the Python surface in process and shells out to the other two. The three surfaces do not run identical workloads, and the table says which is which. The scenario-generation column is the same function emitting the same canonical JSON bytes on all three, so those three numbers are directly comparable. The stepping columns are not. The native row steps a scalar `TradingEnv` through the Rust API with no serialization per

Table 3: Measured throughput per surface on a 4×120 panel, median of three repeats of 500 episodes each. Scenario generation produces the canonical JSON serialization on every surface and is the one directly comparable column. Steps per second and episode wall time reflect each surface's own stepping path, described in the text: the native and Python rows step 120 bars per episode, the WebAssembly row runs a 100-bar baseline episode inside the kernel.

Surface	Stepping path	Scenarios/s	Steps/s	Episode (ms)
Native Rust	scalar `TradingEnv` through the Rust API, no per-step serialization	50,031	747,468	0.161
WebAssembly	baseline episode inside the kernel, one boundary crossing per episode	29,211	508,127	0.197
Python	Gymnasium adapter, JSON `Decision` encoded on every step	44,026	14,240	8.43

step. The Python row steps the Gymnasium adapter, which crosses the binding and encodes a JSON `Decision` on every step, and pays a factor of about 52 for it. The WebAssembly row runs a whole baseline episode inside the kernel with one boundary crossing per episode, so its per-step figure measures the kernel rather than the interface.

The number that matters for a submission is the cost of one protocol-conformant evaluation. Running the full six-policy reference field over the canonical held-out band on all three tiers, 4,608 episodes and 552,960 steps, takes $302.0000\,\mathrm{s}$ of wall clock on the Python surface, the slowest of the three and the one an entrant is most likely to use. That is roughly 1,830 steps per second rather than the 14,240 of the stepping row, because the evaluation figure also carries the reference policies' own computation and the SharpeBench kernel's scoring; a single agent's single-tier evaluation is one sixth of it. A learning baseline is not blocked by the environment either: at the Python surface's measured stepping rate, ten million environment steps cost about $12.0000\,\mathrm{min}$ of environment time, and at the native rate about $13.0000\,\mathrm{s}$, so the compute constraint Section 10 names belongs to the learner rather than to the simulator. Throughput is hardware-dependent, and it is the one class of number in this paper a replicator should expect to differ.

7 Instrument validation

A simulator paper that reports a stylized-facts battery it passes has, in this literature, reported very little. Cont [2001] says so in the founding paper: the stylized facts are "usually formulated in terms of qualitative properties of asset returns and may not be precise enough to distinguish among different parametric models." The objection has been restated at every subsequent methodological turn. Fagiolo et al. [2019] grade empirical validity on four levels, from a Level 0 caricature through qualitative and then quantitative agreement with macro structures to Level 3 quantitative agreement with micro structures, observe that publishing standards now require at least Level 1, and survey the techniques built to "better discriminate among different ABMs reproducing the same set of stylized facts." Lamperti [2018] is blunter, that "moments are poor representations of time series' behaviour": two series can agree on every stylized fact and every moment while their dynamics diverge point for point. Twenty-four years after Cont, LOB-Bench restates the objection for machine learning, that prior work "relied on qualitative comparisons of stylized facts, making rigorous model comparisons infeasible and hindering research progress" [Nagy et al., 2025]. The standard failure mode of this genre is therefore not a simulator that fails its realism check. It is a simulator that passes one and is degenerate anyway, because the check cannot tell models apart.

Read against that, the headline result of this section is evidence about the instrument rather than a shortfall in it. A public, committed, deterministic generator fails a calibrated three-check conjunction on 23 of 24 seeded panels, against an IID null fixed in source and tuned to no market panel, and the failure is reproducible from a command in Section A by anyone who disbelieves it. A gate a generator can fail is a gate with discriminating power, and in this literature that is rarer than a generator that passes an uncalibrated one. We therefore report the result the way Vyetrenko et al. [2020]

Table 4: Stylized-facts results over 8 seeds. Fano is a conditional-IID-calibrated exploratory ratio and has no pass column; the final rate is the conjunction of the three calibrated directional checks, including aggregation toward zero absolute excess kurtosis. Aggregation passes 8/8 seeds in Calm, 8/8 in Hard, and 8/8 in Extreme.

	Excess kurtosis		$\lvert r\rvert$ autocorr		Fano proxy	Gated facts
Tier	mean	pass	mean	pass	mean	pass rate
Calm	-0.97	0/8	$+0.0002$	1/8	1.32	0.000
Hard	$+16.30$	8/8	-0.0107	0/8	0.93	0.000
Extreme	$+11.38$	8/8	-0.0118	1/8	0.91	0.125

and Nagy et al. [2025] report theirs: we name the fact that fails (absolute-return autocorrelation, on all three tiers), name the configurations that fail it (the canonical generator everywhere, and the Calm tier on excess kurtosis besides), report the opt-in driver's relative improvement without promoting it to a realism claim, and locate the contribution in the measuring instrument rather than in the thing measured. Vyetrenko et al. [2020] set the register in one sentence: "there are still significant discrepancies between simulated markets and real ones. However, this work serves as a benchmark against which we can measure future improvement." Platt and Gebbie [2018] reach the same conclusion from the other direction, that "traditional stylized fact-centric validation seems unable to detect these potential problems, suggesting that such methods of validation are simply not sufficient." On the Fagiolo ladder the canonical tape sits at Level 0 on the axis this section gates and makes no claim above it.

Validation runs on four axes, and the section is organized by them, because the environment fails the first and passes structural checks on the other three. A single verdict about "realism" would misreport all four. Section 7.1 is *unconditional* realism, whether the marginal and autocorrelation structure of the tape resembles the classical catalogue. Section 7.2 is *response* realism, whether the price responds to flow the way a no-manipulation model requires. Section 7.3 is *mechanism* realism, whether informed flow costs a maker what the microstructure literature says it should. Section 7.4 is *adversarial* realism, whether a public deterministic generator withstands an adversary who attacks the generator rather than the interface. Within each axis, every follow-up unit sits directly after the finding it extends.

F4 grades the canonical position-trading generator; F5 and F6 run on the market-making and market-clearing models, which are separate simulators with their own structural and positive-control checks and are not submitted to F4, as Section 10 states. Dispersion follows three conventions: t-based 95% intervals over committed per-seed or per-episode vectors for F5 and F6, count-based pass rates for F4, and across-seed standard deviations for the predictability probe, each stated where it is used. Every entry point, seed set and quantity is named where it is reported, the numbers come from JSON and figures committed with their producer, and the commands are in Section A.

7.1 Unconditional realism: stylized-facts certification (F4)

For each tier we grade price panels from eight seeded episodes with `certify_realism` against three directional bounds from the classical catalogue: positive excess kurtosis, absolute-return autocorrelation above a fixed 95th-percentile IID null for this exact 120×4, ten-lag estimator, and aggregation moving *absolute* excess kurtosis toward zero [Cont, 2001]. The IID null is a deterministic 20,000-panel simulation fixed in source and tuned to no market panel. Gain/loss skew [Cont, 2001], timescale asymmetry [Zumbach, 2009], and a finite-panel-calibrated Fano exceedance-count ratio are informational. The Fano ratio conditions on the observed exceedance count and divides the Bessel-corrected sample Fano by its exact equal-block IID expectation; it remains ungated because 12 count windows make it too noisy for certification.

Table 4 and Figure 1 give a mixed verdict. Calm is platykurtic (mean excess kurtosis -0.97), although its returns move closer to Gaussian in absolute-kurtosis distance on aggregation (mean $+0.57$), and

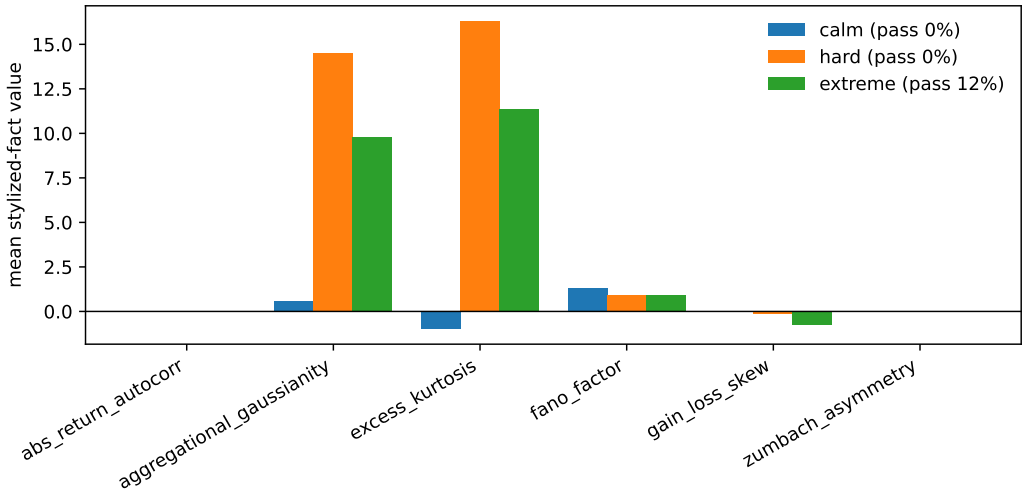


Figure 1: Per-seed stylized-fact values by tier. Only kurtosis, absolute-return autocorrelation and aggregational Gaussianity are gated; Fano is exploratory.

fails the conjunction on all eight seeds. Hard and Extreme pass excess kurtosis and aggregation on all eight seeds, but neither clears the calibrated ACF threshold consistently: Hard passes 0 of 8 and Extreme 1 of 8. The canonical generator therefore passes 1 of 24 seeded panels, and the single binding fact across all three tiers is volatility clustering as the ACF measures it. The Fano ratios are observations and do not decide certification.

The opt-in volatility-clustering driver (`vol_clustering`) adds deterministic EMA persistence to the same seeded panel while preserving the zero-knob golden path. Under the calibrated ACF rule at strength 0.5, the three-check pass rates rise to 0/8, 8/8 and 4/8 for Calm, Hard and Extreme, so the driver moves Hard from failing to passing and Extreme halfway. That is a relative improvement in a named opt-in configuration, and we state it as one: the certified path is the zero-knob canonical generator, and it still fails. The opt-in jump-burst knobs and the exploratory Fano ratio support no remediation claim.

7.1.1 Calibrating Calm with the existing knobs: a negative result

The obvious next question is whether the shipped knobs can be tuned into a Calm tier that passes, so we ask it under a rule fixed before the sweep. We sweep the existing opt-in families, `vol_clustering` in $\{0, 0.3, 0.5\}$ and the jump-burst probability, persistence and size grid, for 99 configurations. The protocol is fixed in the producer: a setting must pass the three-check conjunction on at least 7 of 8 diagnostic seeds, have mean realized-volatility ratio at most 1.25 versus default Calm, and pass at least 7 of 8 disjoint confirmation seeds (100–107); among survivors it would minimize normalized RMS return perturbation. The ACF check uses a one-sided 5% IID-null test for the exact 120×4, ten-lag estimator rather than the raw condition "ACF > 0". No cell satisfies all three predeclared steps, so there is no certified Calm preset and no report-band estimate to promote.

The closest low-volatility candidate is `vol_clustering`$= 0.5$, burst probability 0.02, persistence 0 and size 0.015. It passes 7/8 diagnostic panels but only 5/8 confirmation panels at a mean volatility ratio 1.150. Several settings reach 7/8 on both bands only by exceeding the 1.25 volatility constraint; the strongest near candidate (clustering 0.5, probability 0.02, persistence 0.5, size 0.015) has 8/8 and 7/8 passes but ratio 1.289. We retain the former as a reproducible `calm_calibration_candidate` in the Rust and Python surfaces, pinned by a golden fingerprint, precisely so a future generator revision can be compared against it; it is not a certified configuration (Figure 2).

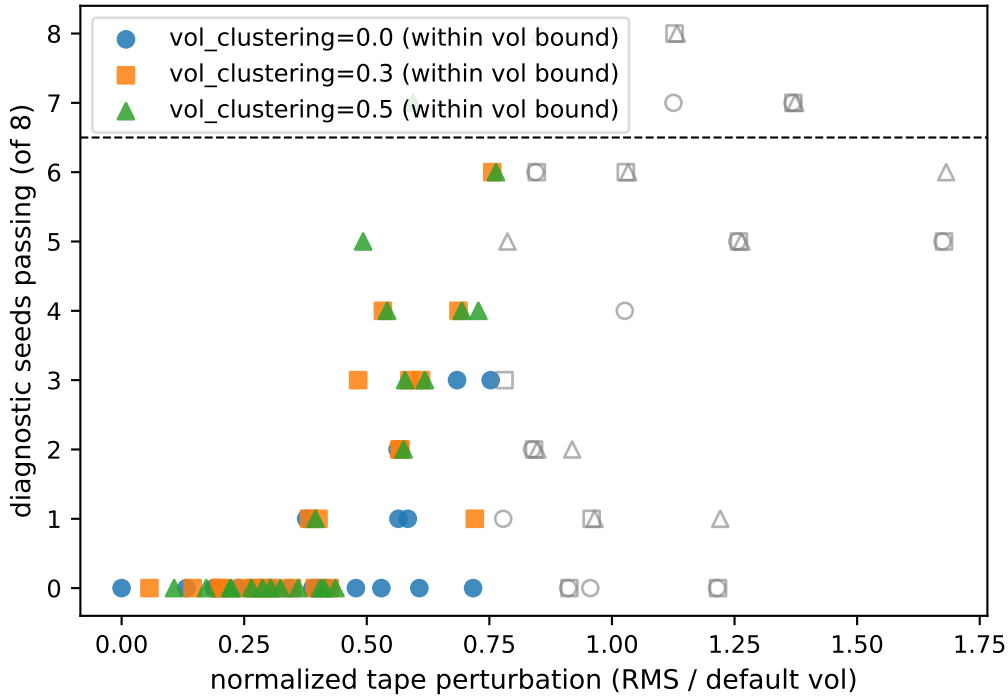


Figure 2: Calm calibration sweep. Each point is one of 99 configurations; the dashed line is 7 of 8 diagnostic passes and hollow points exceed the 1.25 volatility constraint. No configuration meets the full diagnostic–confirmation–volatility rule under the finite-panel-calibrated ACF gate.

7.2 Response realism: the manipulation boundary and size response (F5)

Everything in this subsection is a search over a finite grid of sampled schedules and coefficients. Sampled means are estimates at sampled cells; a negative cell is not a proof over the schedules between the cells, and a positive cell is not an optimum. We state that once here and report cell values plainly below.

We score a push-and-unwind manipulator against its zero-impact paired reference (identical seed and schedule, $\lambda = \eta = 0$) over eight seeds, so `impact_pnl` attributes profit specifically to having moved the price. `impact_boundary_sweep` sweeps permanent impact (Kyle λ), temporary impact (η) and the momentum-follower gain over six grid points per axis; `size_response` sweeps the push weight over five points. The evidence commits per-seed impact-PnL vectors at every grid point; intervals below are t-based 95% CIs over the eight seeds.

Under the precise linear, time-independent permanent-impact assumptions studied by Huberman and Stanzl [2004] and the related no-dynamic-arbitrage conditions of Gatheral [2010], round-trip price manipulation is ruled out, so the linear sweep is a model-consistency check on the environment rather than a discovery procedure. The three units that follow test what a linear symmetric sweep structurally cannot reach.

Table 5 and Figures 3 and 4 are consistent with the linear model's no-manipulation conditions on the sampled grid. On every axis the pump loses money at every sampled coefficient, with every per-point 95% CI excluding zero, including the realistic center of the grid ($\lambda = 0.1$, $\eta = 0.05$: impact PnL -3.4×10^{-4}, CI $[-4.3, -2.6] \times 10^{-4}$). The size response is bounded and decreasing: the least-bad size is the smallest sampled push weight and the loss grows to -5.1×10^{-4} at full size.

Table 5: Manipulation probe, impact-attributable PnL as fraction of capital per episode, in units of 10^{-4}, mean over 8 seeds with t-based 95% CIs. Left: endpoints of each swept axis (6 grid points per axis); every axis is unprofitable everywhere, so no profitability boundary exists to report. Right: the full size-response grid in the push weight.

Axis	Low end	High end	Boundary
Kyle λ (0 to 0.8)	-1.3	-31.7 $[-44.3, -19.2]$	none
η (0 to 0.8)	-2.2	-22.4 $[-23.5, -21.4]$	none
Follower gain (0 to 120)	-2.6	-6.7 $[-10.1, -3.3]$	none

Push wt.	PnL	95% CI
0.10	-0.14	$[-0.25, -0.03]$
0.25	-0.50	$[-0.77, -0.24]$
0.50	-1.53	$[-2.05, -1.00]$
0.75	-3.07	$[-3.84, -2.30]$
1.00	-5.14	$[-6.16, -4.12]$

7.2.1 Normalized-flow concavity: a falsifiable symmetric probe

Empirical impact estimates are concave in traded quantity rather than linear [Almgren et al., 2005], so a probe that can only express linear impact cannot return a positive even in principle. To make it capable of one, the engine supplies an opt-in permanent-impact exponent on dimensionless crowd flow $x = Q/V$:

$$I_\beta(Q; V) = \lambda \operatorname{sign}(x)|x|^\beta, \qquad x = Q/V.$$

The temporary own-size term remains $\eta q_i/V$. This normalization is load-bearing: applying a power to raw shares and dividing by V afterwards would change units with β and make results depend on the arbitrary share denomination. A regression test rescales capital, flow and V together and requires the nonlinear impact fraction to remain unchanged. At $\beta = 1$ the engine uses the original expression $(\lambda Q + \eta q_i)/V$ exactly, preserving every linear golden bit; only $\beta \neq 1$ enters the `powf` path outside the cross-runtime golden guarantee. At fixed λ and sub-unit $|x|$, lowering β also raises the impact magnitude relative to the linear term, so this is a falsifiability stress test rather than a scale-matched comparison of impact shapes.

We rerun the symmetric push-and-unwind sweep at $\beta \in \{0.5, 0.7\}$, eight seeds per point, with paired zero-impact references and pointwise t intervals. No sampled mean is positive. At the canonical calibration, impact-attributed P&L is -8.9×10^{-4} (95% CI $[-19.6, 1.8] \times 10^{-4}$) for $\beta = 0.5$, -13.5×10^{-4} (CI $[-21.2, -5.9] \times 10^{-4}$) for $\beta = 0.7$, and -3.4×10^{-4} (CI $[-4.3, -2.6] \times 10^{-4}$) for the linear arm. For $\beta = 0.5$, 7 of 23 stored sweep slots have intervals crossing zero; repeated canonical slots reduce these to five distinct configurations. At $\beta = 0.7$, 0 of 23 cross zero (Figure 5).

The null identifies the schedule's limitation rather than closing the economic question: profitable constructions under nonlinear permanent impact can be asymmetric, and the next unit searches such shapes.

7.2.2 Exploratory asymmetric-schedule positive control

Concavity makes slicing matter. For positive dimensionless flow $x = Q/V$, the sum of the n per-slice impact increments before the engine's multiplicative price compounding is

$$nI_\beta(Q/n; V) = n^{1-\beta} I_\beta(Q; V),$$

so a slowly accumulated leg and a block liquidation need not cancel when $\beta < 1$. The identity is about the impact function, not the compounded terminal price, so it motivates a positive control without guaranteeing that any finite schedule profits after compounding, temporary costs and follower flow.

We cross five up/down duration ratios, three block fractions, three exponents and three market arms: 135 cells, each on eight seeds. All displayed cell intervals are pointwise. Because the reported winner

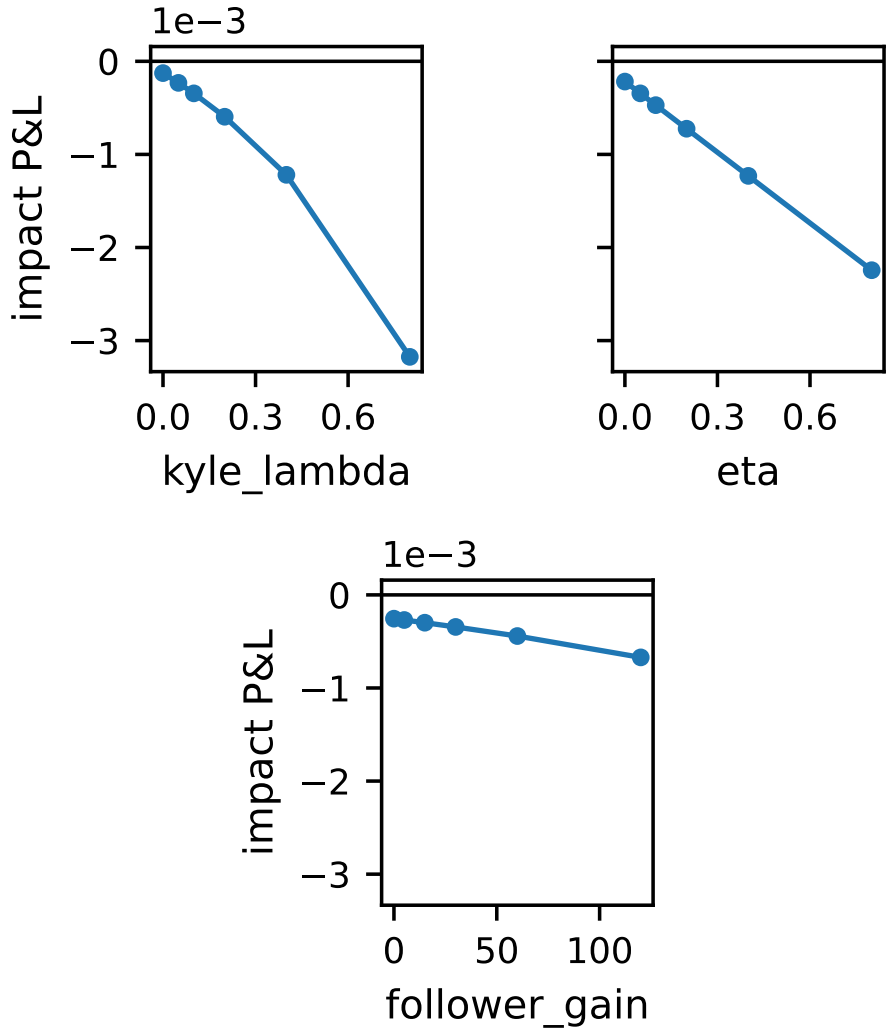


Figure 3: The manipulation probe, boundary sweeps per impact axis: impact-attributed PnL is negative at every sampled coefficient on every axis.

is selected after seeing the grid, its headline inference uses a two-sided Bonferroni familywise 95% interval over all 135 cells (Student t, 7 degrees of freedom); the evidence stores the family size, critical value and adjusted interval.

The best observed cell is the no-follower, zero-temporary-impact arm at $\beta = 0.5$: nine bars of uniform accumulation followed by one-bar liquidation yield $+21.8 \times 10^{-4}$. Its pointwise 95% interval is $[17.4, 26.2] \times 10^{-4}$ and its familywise interval remains positive at $[10.0, 33.6] \times 10^{-4}$. The same sampled schedule with temporary impact yields $+18.4 \times 10^{-4}$, with familywise interval $[6.4, 30.4] \times 10^{-4}$. The canonical follower arm has no pointwise-positive cell. Three granularities of the same count are worth separating: of the 135 cells, four have familywise-positive intervals (the 9:1 and 8:2 uniform schedules on each of the two no-follower arms, $+21.8$, $+10.9$, $+18.4$ and $+8.9 \times 10^{-4}$); they occupy two arms; and the two reported winners are the 9:1 cell on each arm. The fourth of those cells, temporary-impact 8:2, clears zero only at the boundary: its familywise lower bound is $+0.04 \times 10^{-4}$, so a reader recomputing it at a slightly different critical value would count three rather than four. At $\beta = 0.7$ no sampled mean is positive. Among the 45 sampled linear cells,

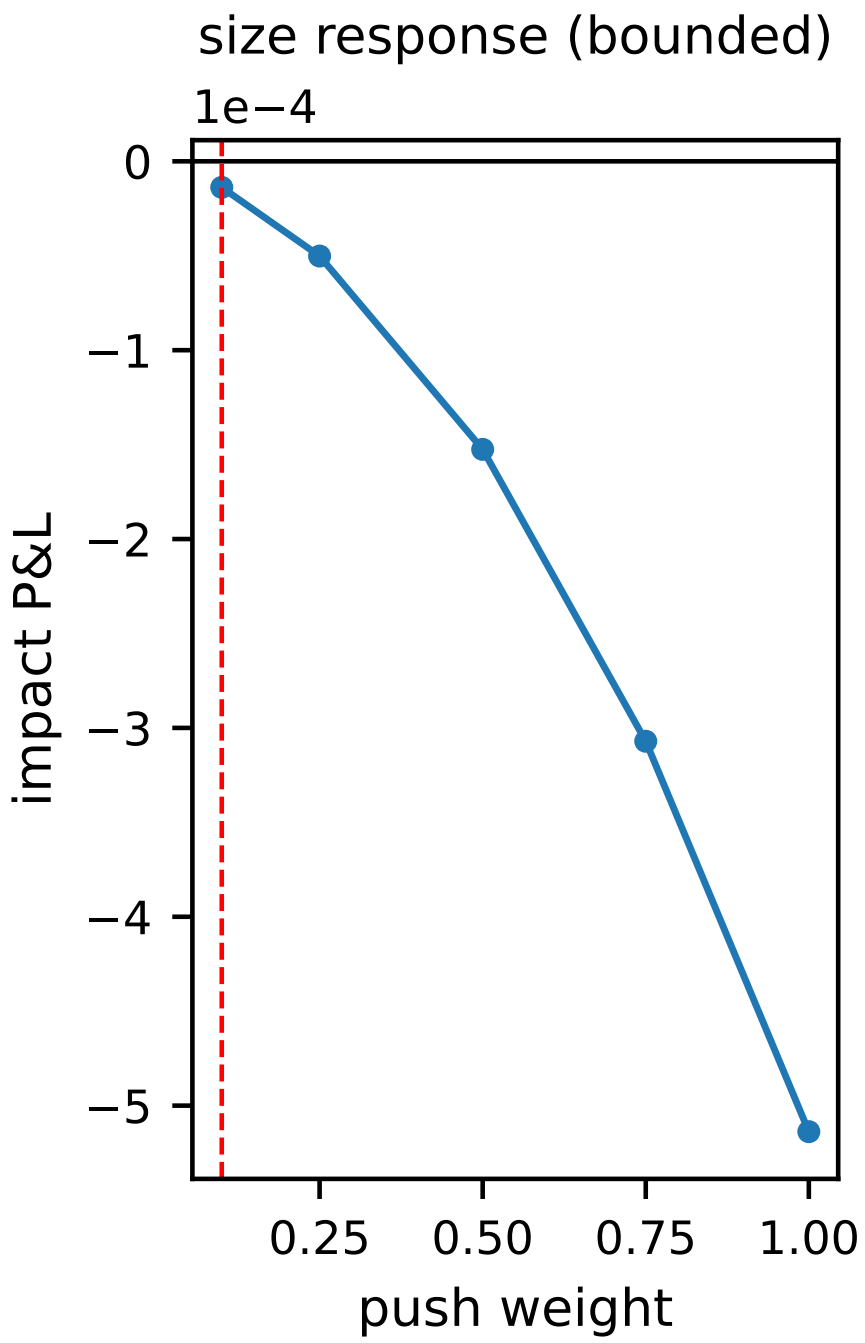


Figure 4: The manipulation probe, size response in the push weight: the loss is bounded and decreasing in size, so the least-bad push is the smallest one sampled.

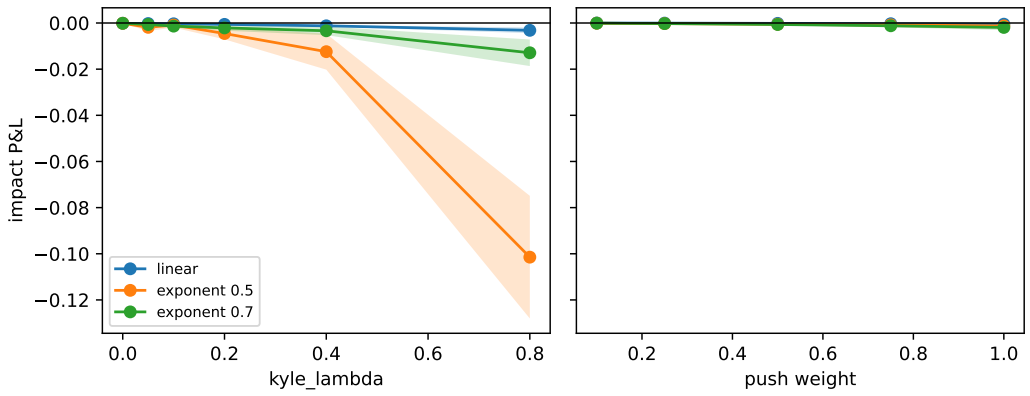


Figure 5: Pointwise impact-attributed P&L intervals for the sampled symmetric schedule under linear and normalized-flow concave impact.

every mean is negative; the 1:9 and 9:1 theory-arm values are numerically near-symmetric rather than bit-identical (Figure 6).

We confirm the selected theory-arm cell independently on 32 fresh seeds fixed before that confirmation run: mean $+26.3 \times 10^{-4}$, two-sided 95% t interval $[23.7, 28.9] \times 10^{-4}$ (df 31). This is one fixed-cell interval and carries no multiplicity correction for the exploratory search, so the 135-cell Bonferroni interval answers the selected-grid question while the fresh-band interval answers the one-cell replication question. What the positive control establishes is that this instrument can detect a profitable sampled round trip under normalized concave impact. What it leaves open is the shape of the profitable region: the search fixes push size, λ, V, total leg length and most follower gains, and omits short-side, overshooting and longer-hold trips. Conversely, the absence of a positive among 45 sampled linear cells is consistent with the cited linear-impact model under its assumptions.

7.2.3 Extended sweeps through the positive-control winner

The positive control leaves six axes at their anchor values, so we sweep each of them one at a time through its no-follower, zero-temporary-impact, uniform 9:1 winner: push weight, λ, total leg length, hold length, short-side trips and follower gain, at $\beta \in \{1, 0.5\}$, five values per axis and eight common seeds, for 60 cells. Because the anchor was selected in the earlier 135-cell grid on these same seeds, the familywise intervals are two-sided Bonferroni 95% t intervals over the global 195-cell family, the antecedent grid plus this extension, rather than over 60 cells alone. Unequal "uniform" legs use separate $1/n_{\text{up}}$ and $1/n_{\text{down}}$ increments, and tests verify that each leg executes exactly one signed peak notional, so changing duration no longer silently changes the amount liquidated.

The results are sharply asymmetric by exponent. Across the 60 extension cells, 23 global-familywise intervals are positive, 31 are negative and 6 cross zero. The longest tested 45:5 theory-arm trip is the largest observed cell, $+135.0 \times 10^{-4}$ with global-familywise interval $[122.1, 148.0] \times 10^{-4}$. Interactions between axes, other exponents, other flow scales, transient impact and overshooting round trips remain untested (Figure 7).

7.3 Mechanism realism: adverse-selection markout (F6)

`compare_informed_vs_uninformed` runs twenty-four paired episodes in which the identical meta-order flow is sided on the alpha in one leg and on a coin flip in the other, holding schedule, sizes and price path fixed, and reports mean maker markout per filled unit at horizons of 1, 5 and 20 bars. Markout is measured in the tape's absolute price units (the mid starts at 100) per unit of filled quantity; the evidence commits per-episode vectors, and the gap CIs below are t-based 95% over the twenty-four paired episodes.

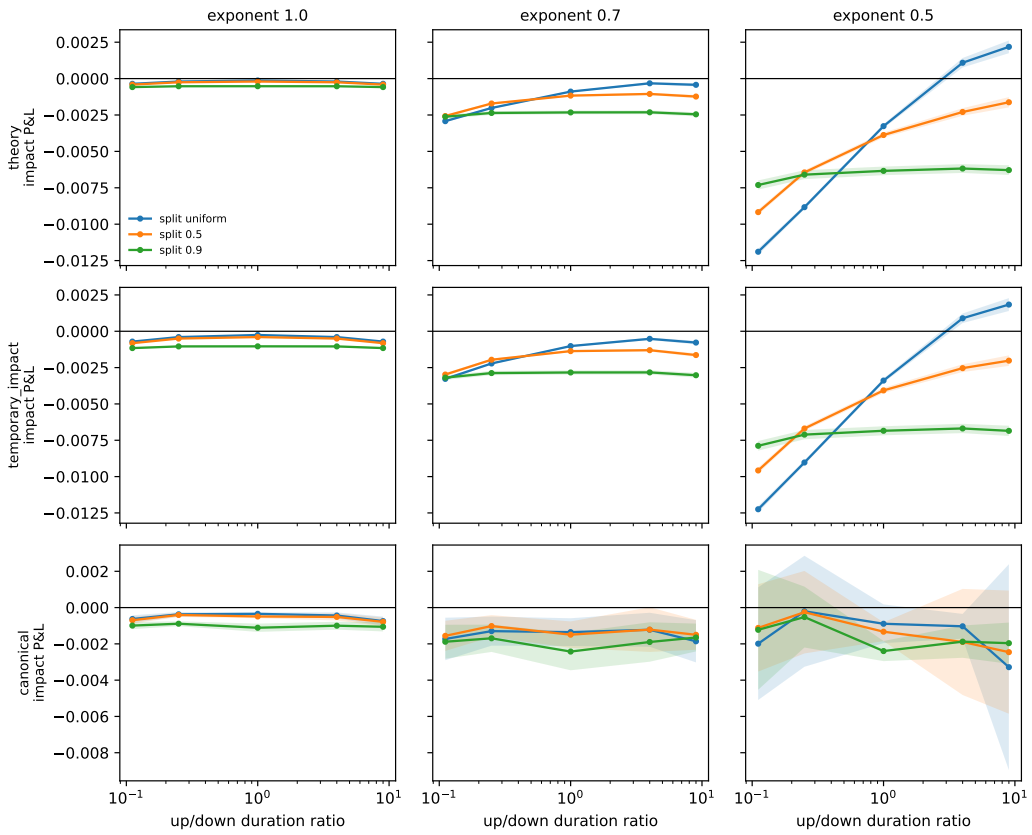


Figure 6: Exploratory asymmetric-schedule search. Shading shows pointwise 95% intervals; selection-adjusted intervals for the two selected $\beta = 0.5$ winners are reported in the text. Four of the 135 cells retain positive familywise intervals, spread over two sampled arms.

Table 6: Maker markout per filled unit (price units per unit quantity) against informed versus paired-uninformed flow, 24 paired episodes; gap CIs are t-based 95% over per-episode paired gaps. The gap is the price of trading against information and grows with horizon; the levels stay positive at every horizon, a separate finding discussed in the text.

Horizon (bars)	Informed	Uninformed	Gap	Gap 95% CI
1	0.689	0.761	0.072	$[0.052, 0.094]$
5	0.489	0.791	0.302	$[0.228, 0.383]$
20	0.386	0.823	0.436	$[0.303, 0.579]$

Two quantities must be separated here: the sign of the *paired estimated gap* and the sign of the *levels*, and they come out on opposite sides.

The paired estimated gap is positive at every horizon (Table 6, Figure 8): makers filled by informed flow mark out worse than the paired-uninformed control at $h = 1$, 5 and 20, the per-unit gap widens from 0.072 to 0.436 as the information realizes, and every horizon's paired CI excludes zero, including the smallest ($h = 1$: $[0.052, 0.094]$ over per-episode paired gaps). The loss is in the drift after the fill rather than in the fill itself, which is the signature shape of adverse selection. The claim is scoped to the paired estimated gap, the differential cost of informed over uninformed flow, and not to maker profitability levels.

The level result goes against the environment, and we report it the way we report F4. Makers earn a *positive* mean markout against informed flow at every horizon (0.689, 0.489, 0.386 per unit), which is on the wrong side of the Section 2 standard that a realistic market should not let makers profit from

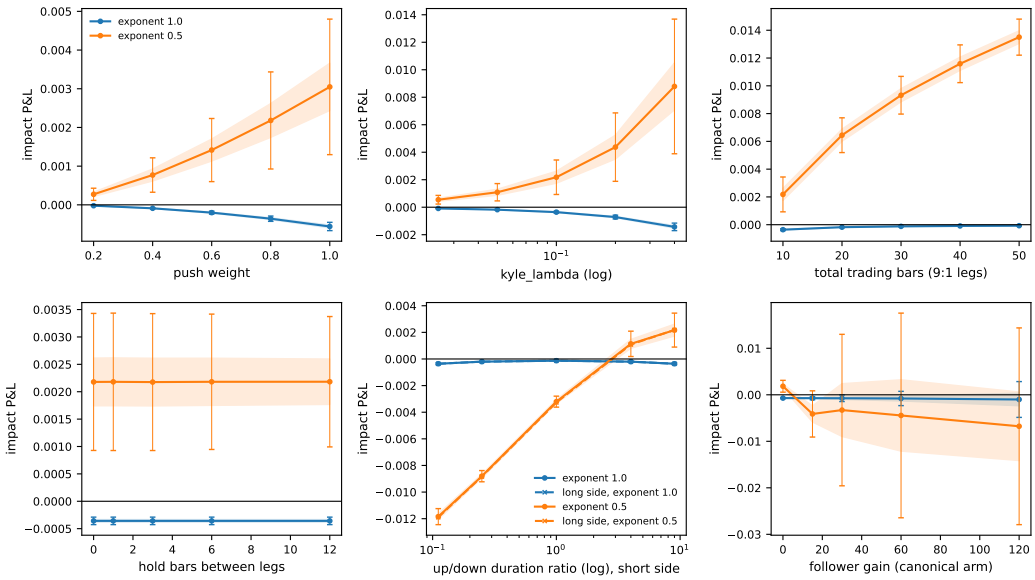


Figure 7: Extended manipulation sweeps after per-leg uniform normalization. Shading is the pointwise 95% interval; error bars are the global 195-cell Bonferroni familywise 95% interval. The displayed maxima are sampled cells, not optimized values.

informed flow. Three calibration choices produce it: fills are struck at the pre-move mid, so every fill books the full quoted depth as spread capture; the default quote depths are wide relative to the drift the $\alpha = 2.0$ meta-orders realize over 20 bars; and the price path is exogenous, so quotes never widen in equilibrium response to toxicity as they would in Glosten and Milgrom [1985]. At this calibration the probe therefore measures markout accounting against a fixed drift, in which informed flow costs makers relative to uninformed flow but not in absolute terms. Recalibrating the informed alpha or striking fills at post-impact prices so the level turns negative, and adding a variant where quotes widen with realized toxicity, are the named follow-ups; the first of them is the subsection below.

The single-episode maker decomposition illustrates the exact identity on one seeded episode (seed 0); it is an illustration, not an aggregate finding. The wider quoter (`maker_0`, 135 fills) keeps a positive markout at $h = 20$ (0.526 per unit, toxic-fill rate 0.296): the flow is not selecting against it. The tighter quoter (`maker_1`, 92 fills) shows the subsidized pattern: aggregate markout is still positive at $h = 20$ (0.197 per unit) but its meta-order fills alone lose 0.197 per unit on 62% of its filled quantity, with a toxic-fill rate of 0.533 at $h = 20$, so noise flow is paying for the informed flow. Spread capture and adverse drift decompose exactly (`spread_capture` + `adverse_drift` = `markout`, in price units times quantity): `maker_1` captures 82.8 of spread and gives back 64.7 to adverse drift by $h = 20$.

7.3.1 The endogenous arm: maker markout when the flow moves the price

The design above fixes the efficient-price path, so it measures the informed-versus-uninformed *gap* but cannot speak to maker profitability *levels*: a maker whose fills never move the price against it books the full quoted depth on every print, and the positive levels reported above are partly that artifact. This arm makes the level question answerable. We rerun the same twenty-four paired episodes with the cleared price endogenous: the taker flow actually filled against the maker ladder moves the mid through the permanent-impact law of the shared-book clearing engine of Section 3. Let S_t be the efficient price (the meta-orders' alpha increments plus ordinary volatility, identical across both legs and both arms), Q_t the signed taker quantity filled in bar t (meta-order children plus noise flow, buys positive), λ Kyle's permanent-impact coefficient [Kyle, 1985] and V the engine's

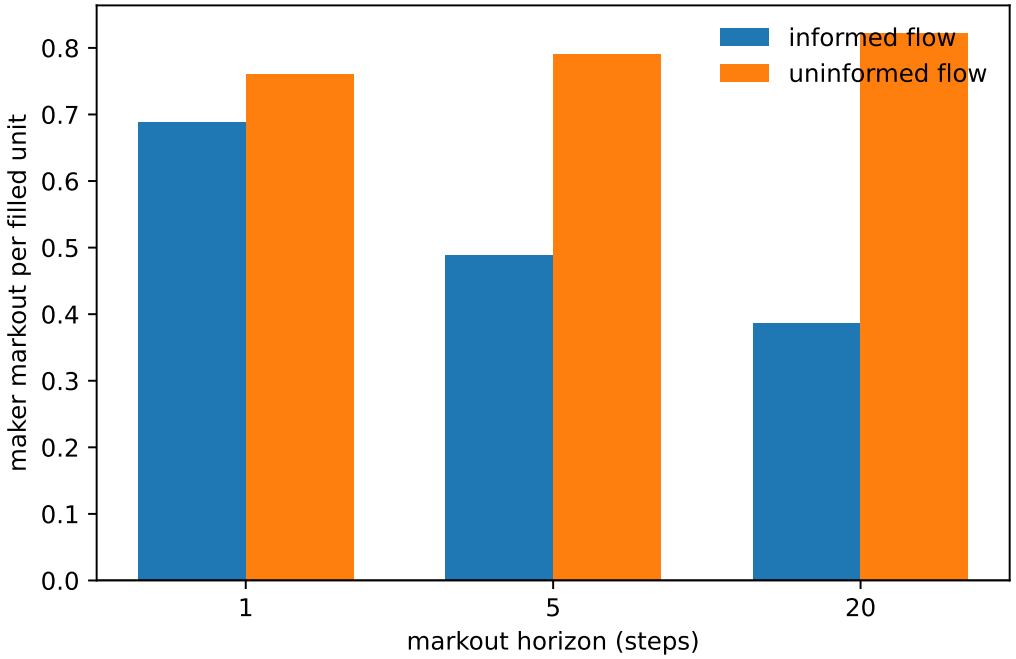


Figure 8: Markout per filled unit by horizon: informed flow versus the paired-uninformed control.

ADV-like volume normalizer. Makers then quote around

$$\mathrm{mid}_t \;=\; S_t\, M_t, \qquad M_{t+1} \;=\; M_t\big(1+\lambda\, Q_t/V\big), \qquad M_0 = 1, \tag{1}$$

the engine's running multiplier, updated after the bar clears so the reference price a maker quotes against is moved only by flow strictly before it. Temporary impact is not a separate term: the maker ladder itself is the temporary component (a taker pays the quoted depth, which is what the engine's Almgren–Chriss η term models in the maker-less market [Almgren and Chriss, 2001]), so the arm adds exactly one mechanism to the exogenous scenario, that a filled unit moves the next reference mid by $\mathrm{mid}_t\,\lambda/V$ in the taker's direction. At $\lambda = 0$ the arm reproduces the exogenous scenario bit for bit, and a cross-check in the test suite feeds the identical flow sequence to the native clearing engine and matches its cleared mids to relative 10^{-12}, so Equation (1) is the engine's law and not a paraphrase of it. The calibration sets $V = 240$, one episode's realized taker volume (mean 239 filled units over the 48 exogenous-arm episodes), and the engine's default $\lambda = 0.1$, so a filled unit displaces the mid by 0.042 price units at the initial mid of 100; a parent order fills a mean 22.4 of its 60 offered units, displacing the mid by a mean 0.81% over its own window (t-based 95% CI $[0.77, 0.85]\%$ over episodes, sided with the parent in both legs, and zero by construction in the exogenous arm).

Both effects the design predicts appear (Table 7, Figure 9). First, the gap moves by at most 0.011 per unit: 0.072, 0.300 and 0.430 at $h = 1$, 5 and 20 against 0.073, 0.305 and 0.441 in the exogenous arm, every CI excluding zero. Permanent impact is direction-agnostic from the maker's seat, since the maker is on the wrong side of the move its own fill causes whichever way the taker trades, so it cancels out of the informed-minus-uninformed difference; the gap is the information, and the arm confirms it is not an artifact of the fixed path. Second, the levels fall in both legs at every horizon: informed markout drops from 0.385 to 0.240 per unit at $h = 20$, uninformed from 0.827 to 0.670, and the informed toxic-fill rate rises from 0.381 to 0.437. That drop is the impact cost the exogenous design could not book.

The economically meaningful question is whether makers still profit against informed flow once their own fills move the price, and at this calibration the answer is yes: the informed level at $h = 20$ is 0.240 with pointwise CI $[0.192, 0.289]$, positive but 0.145 per unit lower than the exogenous arm's.

Table 7: Maker markout per filled unit (price units per unit quantity) under the exogenous and endogenous price paths, 24 paired episodes, horizons in bars. Every cell is the mean over per-episode values (episode markout over episode filled quantity, all makers; toxic rate is the share of all maker fills marking out negative), with t-based 95% CIs, $\mathrm{df} = 23$; this is the convention of the committed per-episode vectors and of the gap CIs in Table 6, so the exogenous rows differ from the pooled quantity-weighted aggregates of Table 6 in the third decimal. The evidence also serializes paired endogenous-minus-exogenous changes and their CIs. Endogenous arm at $\lambda = 0.1$, $V = 240$.

Path	h	Informed	Uninformed	Gap	Toxic-fill rate (inf / uninf)
exogenous	1	0.689 [0.683, 0.695]	0.762 [0.740, 0.783]	0.073 [0.052, 0.094]	0.016 0.044
exogenous	5	0.488 [0.470, 0.507]	0.794 [0.720, 0.867]	0.305 [0.228, 0.383]	0.221 0.157
exogenous	20	0.385 [0.341, 0.430]	0.827 [0.711, 0.942]	0.441 [0.303, 0.579]	0.381 0.279
endogenous	1	0.622 [0.614, 0.629]	0.693 [0.672, 0.715]	0.072 [0.051, 0.093]	0.034 0.056
endogenous	5	0.352 [0.327, 0.376]	0.652 [0.579, 0.724]	0.300 [0.225, 0.375]	0.323 0.209
endogenous	20	0.240 [0.192, 0.289]	0.670 [0.558, 0.782]	0.430 [0.294, 0.565]	0.437 0.297

Table 8: Endogenous-arm markout per filled unit at $h = 20$ as the permanent-impact coefficient sweeps, $V = 240$ throughout, the same 24 paired episodes, and exploratory pointwise t-based 95% CIs ($\mathrm{df} = 23$); no sweep-wide multiplicity claim is made. "Impact/unit" is the mid displacement per filled unit at the initial mid, $100\,\lambda/V$. The $\lambda = 0$ row is the exogenous arm.

λ	Impact/unit	Informed	Uninformed	Gap	Toxic (inf)
0.00	0.000	$+0.385$ $[+0.341, +0.430]$	$+0.827$ $[+0.711, +0.942]$	0.441 [0.303, 0.579]	0.381
0.05	0.021	$+0.313$ $[+0.267, +0.359]$	$+0.748$ $[+0.635, +0.862]$	0.435 [0.299, 0.572]	0.415
0.10	0.042	$+0.240$ $[+0.192, +0.289]$	$+0.670$ $[+0.558, +0.782]$	0.430 [0.294, 0.565]	0.437
0.20	0.083	$+0.095$ $[+0.039, +0.152]$	$+0.513$ $[+0.403, +0.623]$	0.418 [0.282, 0.554]	0.479
0.30	0.125	-0.050 $[-0.117, +0.017]$	$+0.356$ $[+0.245, +0.467]$	0.406 [0.268, 0.544]	0.507
0.40	0.167	-0.196 $[-0.275, -0.117]$	$+0.198$ $[+0.084, +0.312]$	0.393 [0.251, 0.536]	0.525

The sweep in Table 8 is exploratory. Its sampled informed level declines from $+0.095$ at $\lambda = 0.2$ to -0.050 at $\lambda = 0.3$ (whose pointwise interval crosses zero) and -0.196 at $\lambda = 0.4$, while the uninformed level remains positive over these six values and the paired gap changes from 0.441 to 0.393. The data therefore bracket an observed sign change between sampled values; they do not identify a crossover at $\lambda \approx 0.27$ or attach uncertainty to an interpolated value. The default calibration ($\lambda = 0.1$) is one explicitly sampled point below that bracket.

The impact model is the engine's stylized linear permanent impact with a single volume normalizer rather than an estimated impact curve, and linearity is itself a modeling commitment [Huberman and Stanzl, 2004]; the temporary component is folded into the maker ladder rather than carried as a separate η term; fills are still struck at the maker's quote around the pre-move mid; and the quoting side is one follower model, the same fixed Avellaneda–Stoikov-plus-fixed-spread roster as above, which never widens in response to realized toxicity, so this is still not the equilibrium quote response of Glosten and Milgrom [1985]. What the arm establishes is narrower: on this sampled calibration grid, with the same quotes and information, informed-flow maker profitability declines as flow impact increases, while the paired estimated gap changes modestly. The exogenous and endogenous arms share one code path and one artifact, pinned by a golden hash asserting that the exogenous reports do not move when the endogenous arm runs; the endogenous evidence, per-episode vectors and paired arm changes included, lives under the `endogenous` key of the same artifact.

7.4 Adversarial realism: the in-band inversion attack

Section 10 names a threat the leak-free interface property does not cover: a scenario is a pure function of a public 64-bit seed under a public deterministic transform, so future bars are in principle computable from past bars, and no forbidden operation is needed. This probe converts that named threat into a measurement. Three adversaries predict the next bar's return from a causal prefix only, on the three volatility tiers, 16 seeds per tier (4 symbols, 120 days, 30 warmup bars): an *unconditional*

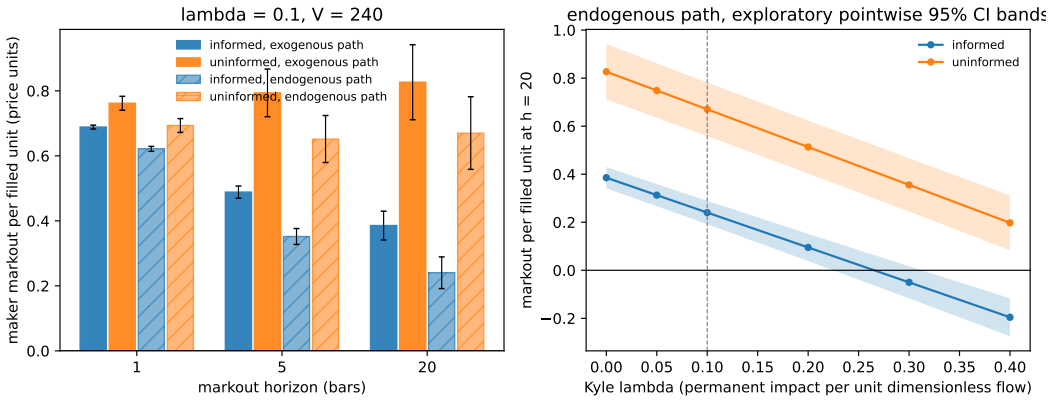


Figure 9: The endogenous arm of F6. Left: maker markout per filled unit by horizon, informed versus paired-uninformed flow, exogenous versus endogenous path ($\lambda = 0.1$, $V = 240$), with t-based 95% CIs over 24 paired episodes; paired arm changes are serialized in the evidence. Right: endogenous markout per filled unit at $h = 20$ across an exploratory six-value λ sweep, with pointwise 95% CI bands. Dashed line: the default calibration.

baseline that predicts the expanding prefix-mean return per symbol; an *honest statistical adversary*, a ridge AR(5) refit on the prefix at every bar, with no knowledge of the generator; and a *known-seed oracle*, which replays the generator from the true seed and therefore predicts every future bar exactly, the upper bound on any inversion attack. We also run each predictor as a frictionless unit-gross long/short sign-following policy scored through the public `score_run` kernel, and report the kernel's deflated-Sharpe probability.

The honest adversary extracts almost nothing. All "$\pm$" quantities here are across-seed standard deviations over the 16 seeds, computed with the population convention (`ddof` 0), not confidence intervals. On Calm the ridge AR reaches $55.4\% \pm 2.8\%$ directional accuracy against the baseline's $54.1\% \pm 2.4\%$: it detects the tier's designed momentum autocorrelation, worth about one descriptive accuracy point over drift-following. On Hard and Extreme the AR accuracies are $51.2\% \pm 3.3\%$ and $50.6\% \pm 2.1\%$, with MSE no better than the unconditional baseline (Hard: 1.24×10^{-4} vs. 1.22×10^{-4}; Extreme: 1.27×10^{-3} vs. 1.17×10^{-3}). No hypothesis test or equivalence margin was preregistered, so these summaries support only the descriptive statement that the adversary lands near the unconditional baseline. One post-hoc check is worth reporting rather than hiding: a paired t-test of the ridge AR against the baseline over the same 16 Calm seeds gives $t(15) = 3.36$, $p = 0.004$, so the one-point Calm edge over drift-following is larger than seed noise, exactly as the tier's disclosed momentum structure predicts; the same test is inconclusive on Hard ($p = 0.08$) and Extreme ($p = 0.40$).

The oracle bound is total, and a bounded seed search reaches it. The known-seed oracle scores 100% accuracy, zero MSE, and DSR 1.00 on every tier: determinism means the gap between an honest adversary and a successful inverter is the entire distance from coin flipping to omniscience, so the question is only the cost of inversion. We implement the strongest cheap version, a brute scan over a bounded seed band of width 2^{16} centered on the published `start_level`, matching each candidate's first-bar closes against the single observed opening bar. The scan recovers the true seed in 16 of 16 trials with zero collisions, from a prefix of one bar, at $14.7\,\mu$s per candidate (≈ 0.96 s per scan); every recovered seed then reproduces the full 30-bar observed prefix on its tier. When a deployment uses a bounded public `(start_level, num_levels)` specification, this in-band inversion is a table scan rather than cryptanalysis, and its scenarios should be treated as known to a motivated adversary (Figure 10). Extrapolating this Python harness's measured per-candidate cost to the unbounded 2^{64} space gives ~ 8.6 million CPU-years. The figure is harness-specific: a native

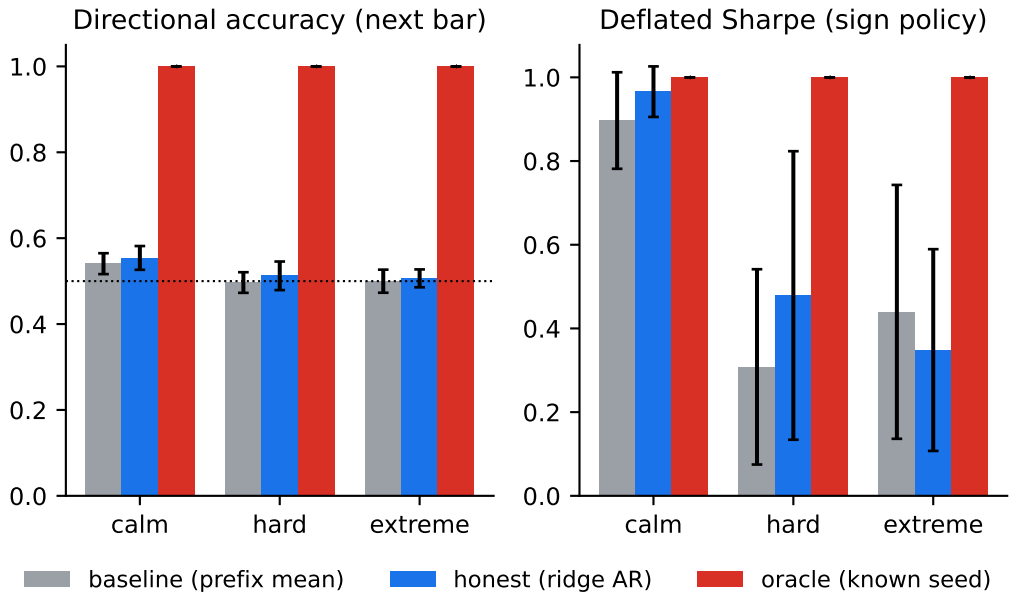


Figure 10: Next-bar directional accuracy by tier. The causal AR adversary remains near the unconditional baseline; knowing or recovering the seed gives perfect prediction.

scanner would be orders of magnitude cheaper per candidate, which moves the number but not the conclusion that brute-force inversion of an unpublished 64-bit seed is infeasible by enumeration.

What remains open. The probe brackets the attack but does not close it. Between the honest regression (no edge beyond disclosed structure) and the band scan (total recovery given a bounded public band) lies the case that matters for unbounded deployments: analytic inversion of SplitMix64 [Jr. et al., 2014] through the mul/add/div/max price transform from observed reals. We implement and test the narrow algebraic step: SplitMix64's 64-bit finalizer is exactly invertible when its complete output word is known. The generator's `next_unit`, however, exposes only the upper 53 bits as a float, leaving 2^{11} candidate post-increment states from one exact unit observation before any price arithmetic is considered. The public interface reveals neither that unit nor an unrounded finalizer output: it reveals prices after data-dependent arithmetic, floors and serialization. Deciding among those candidates from observed prices remains the untested cryptanalytic step; the new primitive reduces the question without claiming a tape-level break. This probe therefore supports only the narrower conclusion that resistance to this attack rests on seed secrecy and entropy rather than transform hardness, and that published bounded seed bands forfeit both.

7.4.1 Sealed evaluation seeds: defeating the bounded-band scanner

The public held-out set is disjoint from training but enumerable, so the band scanner above recovers its scenario identities without violating the observation interface. The opt-in mitigation derives each named slot from a secret salt k:

$$\text{seed}_i = \texttt{EVAL\_SEED\_BASE} + \big(F(k,i) \bmod (2^{64} - 1 - \texttt{EVAL\_SEED\_BASE})\big).$$

The shipped F combines FNV-1a/64 with public SplitMix64 finalizers. It is deterministic and pinned across Rust and Python, and the tested salts and slots produce held-out-band seeds distinct from the public set. No universal injectivity claim is possible for a finite 64-bit mapping, and this construction is PRF-style rather than a certified MAC.

The intended workflow is commit–reveal, as Section 6.3 prescribes it. Before evaluation, the host publishes SHA-256(k) and withholds k; after scoring, it reveals k so anyone can recompute the

named seeds and replay the run. SharpeBench's forward-window format can bind that hash before entries arrive. This paper's evidence is a simulated workflow containing the revealed salt, not proof that any live evaluation used secret custody, a non-reuse policy or a reveal deadline.

On 16 tested slots, the scanner over the public 2^{16} band recovers 16/16 public seeds and 0/16 sealed seeds while the salt is withheld; revealing the salt replays 16/16. Under a uniform-seed approximation, that scanner covers about 3.6×10^{-15} of the derived band, so its expected number of recoveries over 16 slots is 5.7×10^{-14}: the 0/16 outcome is what the coverage arithmetic already implies, and the run verifies the implementation rather than testing an uncertain hypothesis. The result is therefore narrow: high-entropy salt secrecy defeats this particular bounded-band enumeration budget. It is not a cryptographic security proof, and it does not answer tape-level inversion after observations begin.

8 Calibration findings

The previous section audits the environment. This one runs policies through it and reports what the resulting instruments read. Five findings follow: the baseline board under the corrected scoring kernel and the eligibility witness that answers the question the empty board raises (Sections 8.1 and 8.1.1), regret against the closed-form market-making reference (Section 8.2), the generalization gap and cross-regime transfer matrix (Section 8.3), the failure-mode distribution (Section 8.4), and the replicated ecology probe (Section 8.5). Every follow-up unit sits directly after the finding it extends.

F1, F3, F7, F8 and the witness run on the canonical position-trading generator, whose tape the realism diagnostic of Section 7.1 declines to certify on 23 of 24 seeded panels; F2 runs on the Avellaneda–Stoikov market-making model instead. Every number below is therefore a calibration of an instrument on tape the paper does not claim is market-realistic, and Section 10 carries that constraint forward into every claim built on it. The byte-identity guarantee covers the engine outputs behind F1 through F3 and the canonical linear path; the Python evidence reductions are seed-deterministic on one platform but sit outside the golden-hash path. F1 and F3 report 2,000-resample seed bootstrap intervals, F2 reports t-based 95% intervals over committed per-episode vectors, and F7 and F8 are count-based, reporting episode counts and seed-level winner distributions rather than interval estimates. Every entry point, seed set and quantity is named where it is reported, and the commands are in Section A.

8.1 F1: Pipeline validation under the corrected kernel

We roll the reference-policy field (flat, equal-weight long, momentum, minimum-variance, maximum-Sharpe, fractional-Kelly volatility targeting) over sixteen scenario seeds in each of the Calm, Hard and Extreme tiers via `run_baselines`. The SharpeBench `score_run` kernel scores each policy's pooled return series with the declared trial count equal to the field size, and each row carries a seed-paired bootstrap 95% confidence interval on the deflated Sharpe (2,000 resamples).

This experiment validates the producer-scorer pipeline end to end; the scorer-side discovery it reproduces belongs to the companion paper. Under the pre-0.5.0 kernel this board was all zeros: every baseline scored a deflated Sharpe of 0.0000 on every tier, which reflected a unit error rather than strictness. The kernel applied its annualized 0.5 deflation prior per period without conversion, which set the bar near an annualized Sharpe of 18 on daily bars, a level nothing clears. Finding and fixing that unit bug is the companion SharpeBench paper's Finding 1 [Toca, 2026], corrected by converting once at 252 periods per year; what this paper claims is only that the producer-scorer separation surfaced it, because trajectories produced here could be re-scored under both kernels and the discrepancy attributed to the scorer alone.

Table 9 and Figure 11 show the corrected board. On Calm, drift is free deflated Sharpe: `kelly_vol_target` and `equal_weight_long` saturate the DSR at 1.0000, with bootstrap CIs $[0.9798, 1.0000]$ and $[0.8343, 1.0000]$. `min_variance` reaches 0.9849 on a near-vacuous interval of $[0.0264, 1.0000]$. The deflated Sharpe is a probability bounded in $[0, 1]$, so 1.0000 is its ceiling rather than an unbounded score, and the percentile bootstrap undercovers at that boundary: the

Table 9: Baseline leaderboard under the corrected kernel: deflated Sharpe (DSR) and pass^k rate per tier, 16 seeds each. No policy is rank-eligible on any tier: eligibility requires the per-run gate to pass on every seed (a pass^k rate of exactly 1.00), and the best rate is 0.75.

	Calm		Hard		Extreme	
Policy	DSR	pass^k	DSR	pass^k	DSR	pass^k
kelly_vol_target	1.0000	0.75	0.0277	0.31	0.0029	0.12
equal_weight_long	1.0000	0.62	0.1114	0.31	0.0234	0.06
min_variance	0.9849	0.44	0.0024	0.19	0.0243	0.06
flat	0.0007	0.00	0.0007	0.00	0.0007	0.00
momentum	0.0000	0.00	0.0000	0.00	0.0000	0.06
max_sharpe	0.0000	0.06	0.0000	0.00	0.0000	0.00

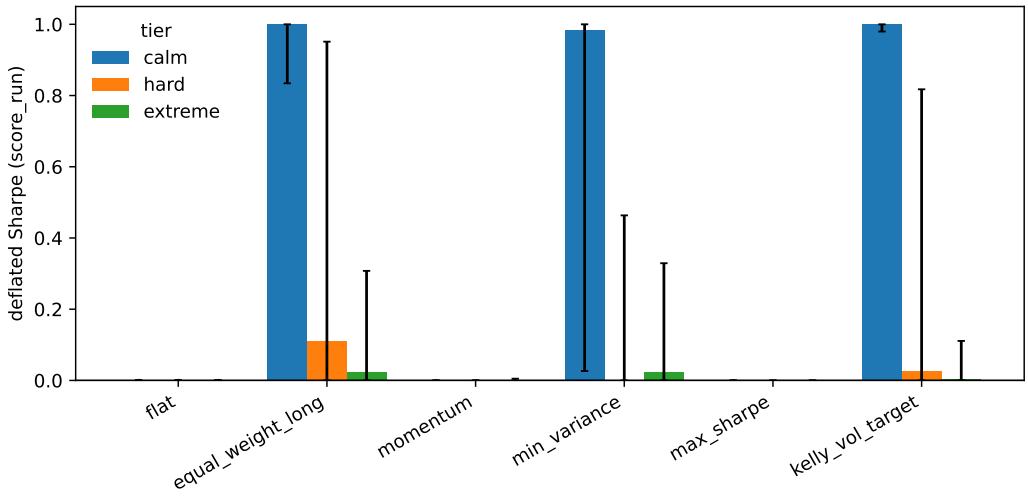


Figure 11: Deflated Sharpe per baseline per tier with seed-paired bootstrap CIs. Calm saturates the long baselines; Hard and Extreme collapse them toward zero.

saturated intervals are indicative, and no bias-corrected variant is applied. Yet nothing on the board is rank-eligible, because no policy passes the per-run gate on every seed; the best pass^k rate is 0.75. On Hard the best DSR collapses to 0.1114 and on Extreme to 0.0243, and the long baseline's pass^k rate degrades monotonically with stress (0.62, 0.31, 0.06). Each rate is 16 Bernoulli seeds, so near a rate of one half a single rate carries a nominal 95% half-width of about 0.24, and a difference between two of them about 0.35: only the Calm-to-Extreme drop of 0.56 exceeds that, so the monotone ordering describes these seeds rather than a resolved ranking. `momentum` and `max_sharpe` lose money outright on every tier (mean per-bar returns down to -1.6×10^{-3} on Extreme).

Eligibility is a conjunction, and each leg catches what the other misses. Deflation alone would crown drift on Calm; pass^k alone would miss the overfit breadth the deflation discounts. The number a real agent has to beat is a positive deflated Sharpe that survives every evaluation seed, and no reference policy produces one. The board alone does not show that the eligibility set is non-empty for this task family. Section 8.1.1 answers that by construction, injecting an out-of-band oracle signal of controlled strength and locating the strength at which the gates first open; the eligibility set is non-empty on every tier, and the per-run gate that fails every baseline here is the leg that binds there too.

8.1.1 Rank-eligibility witness: the eligibility set is non-empty

The baseline board above contains no rank-eligible policy, leaving open whether the conjunction can be satisfied at all. We answer by construction with an out-of-band oracle rather than an entrant. For each seed, the script replays the deterministic, policy-invariant tape one bar ahead and mixes

Table 10: Observed eligibility crossing in signal-truth correlation s: mean [min, max] of the bisection-bracket midpoint over five independent noise paths per cell. The min to max range is variation across noise realizations; the bisection bracket around each individual crossing has half-width 0.0016 and is not shown. Crossings are found by bisection after a monotone coarse-grid check on every path, and are not global optima or theorem-level thresholds. "none (0/5)" means that no sampled $s \leq 1$ is eligible on any of the five paths. At every one of the 50 attained crossings, the last failing gate is the requirement that every seed's marginal PSR reach 0.90.

Variant	Band	Calm	Hard	Extreme
`sign_follow`	gap-band witness	none (0/5)	0.730 $[0.702, 0.752]$	0.343 $[0.302, 0.389]$
`sign_follow`	F1 table	none (0/5)	0.783 $[0.764, 0.820]$	0.353 $[0.317, 0.445]$
`deadband_hold`	gap-band witness	0.848 $[0.777, 0.930]$	0.500 $[0.408, 0.623]$	0.337 $[0.283, 0.383]$
`deadband_hold`	F1 table	0.690 $[0.620, 0.758]$	0.621 $[0.483, 0.755]$	0.342 $[0.283, 0.398]$

standardized next-bar truth z_t with a seeded noise path ε_t:

$$\text{signal}_t(s) = sz_t + \sqrt{1 - s^2}\,\varepsilon_t, \qquad 0 \leq s \leq 1.$$

Within one noise path the same ε_t is reused at every s (common random numbers), so changing signal strength does not silently change the experiment. A sign-following policy trades every bar; a deadband policy trades only when $|\text{signal}| > 1$ and otherwise holds. Both are scored through the unchanged SharpeBench conjunction over 16 seeds. The per-seed gate uses the probabilistic Sharpe ratio (PSR), the probability under the kernel's stated sampling model that Sharpe exceeds its reference threshold. The script records the full coarse eligibility set, the sampled strengths at which the kernel conjunction accepts. It refines an *observed crossing bracket* by bisection only when that coarse set is monotone; otherwise it reports the observed set without calling any crossing a threshold. A crossing located under one noise path is a functional of that draw, so we repeat the whole procedure (coarse sweep, monotonicity check, bisection) under five independent noise paths per cell: the first is the path serialized as the primary witness, the other four redraw ε_t. Each bisection bracket is numerical resolution on the stated grid (half-width 0.0016 in s); the spread of the crossing across noise paths is a separate, statistical quantity, and it is what Table 10 reports. All 50 attained coarse series (10 cells, five paths each) are monotone.

The result is an existence proof and a calibration. The deadband construction is eligible on every tier on every noise path, so the observed eligibility set is non-empty. The every-seed gate is last to open at all 50 observed crossings; pooled DSR and bootstrap gates open earlier. The crossing itself is not a constant of the tier: across noise paths it moves by 0.05 to 0.27 in s (a factor of 15 to 85 above the bisection resolution), widest for the deadband policy on Hard, so any single-path threshold is one draw from that range and the table should be read at the precision of the range rather than of the bracket. Differences between the two bands are differences between two 16-seed scenario bands as well as between noise draws, and with five paths per cell we do not test them. The sign follower never becomes eligible on Calm, on any of the five paths, because its every-bar turnover costs more than the low-volatility signal is worth; at $s = 1$ the noise term vanishes, so that endpoint is noise-invariant by construction, and the failure at every $s < 1$ is what the replication adds. The environment therefore imposes no activity requirement, but its cost model does require a Calm policy to ration turnover (Figure 12).

8.2 F2: Regret against the closed-form reference policy

On the Avellaneda–Stoikov market-making task, `mm_regret` runs each candidate quoting policy and the closed-form reference policy (Section 3.6) on the same sixteen seeded episodes ($\sigma = 2.0$, $\gamma = 0.1$, $\kappa = 1.5$, 200 steps) and reports the mean reward gap to the reference. Candidates are the reference itself and fixed-spread quoters over a grid of half-spreads. The evidence commits the per-episode regret vector per grid point; intervals are t-based 95% CIs over the sixteen paired episodes.

The reference's regret is 0.0 by construction, which confirms the plumbing. The fixed-spread curve is U-shaped (Figure 13): regret is 84.6 (95% CI $[68.2, 101.1]$) at a half-spread of 0.05 (quoting too

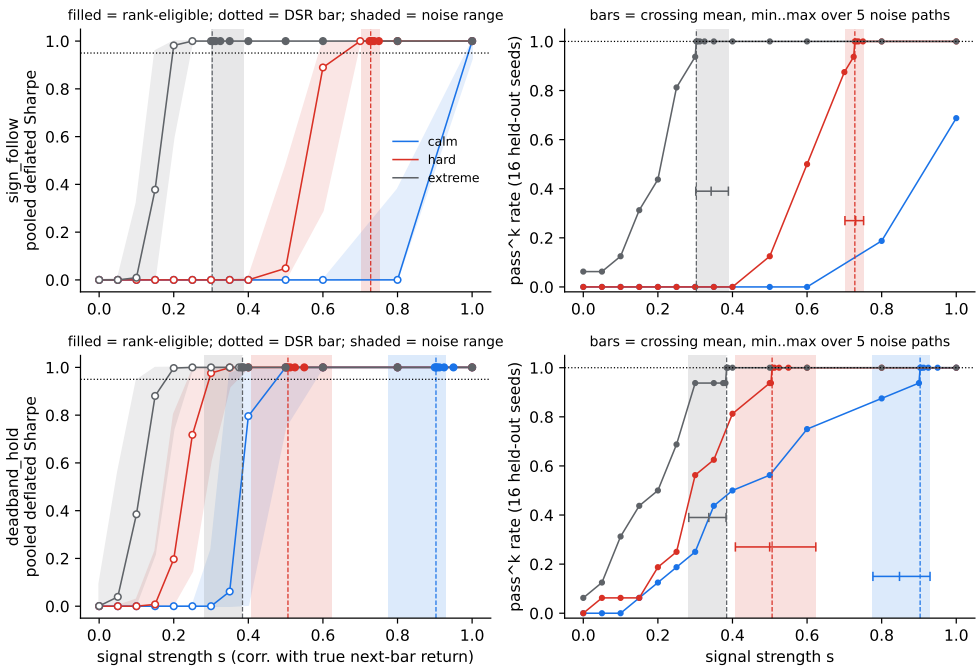


Figure 12: Witness sweep on the gap-band witness subset under the primary noise path. Filled markers are eligible sampled points; dashed lines mark the primary-path bisection crossing, drawn only after monotonicity is verified. Shaded bands (both panels) and the horizontal bars (right panels) give the min to max range and the mean of the crossing over five independent noise paths. The right panels show how many of the 16 seed-level PSR tests pass.

tight, filled constantly, run over by inventory), falls to a minimum of 0.037 at 0.5, and rises again to 36.6 ($[33.0, 40.2]$) at 2.0 and 58.8 ($[55.3, 62.3]$) at 4.0 (quoting too wide, earning nothing). The minimum is the point the intervals do not resolve: the 0.5 point's CI is $[-8.7, 8.8]$ and the 1.0 point's is $[-5.4, 12.4]$. Both intervals include zero, so sixteen episodes do not resolve either point from the analytical reference; this failure to reject zero is not evidence of equivalence. At the observed per-episode dispersion, sixteen paired episodes detect a mean regret of about 12 reward units at 80% power and a two-sided 5% level, roughly a third of the 36.6 measured at a half-spread of 2.0, so differences smaller than that are outside this design's reach. The tight and wide extremes are separated from zero by wide margins. The instrument resolves the shape of the curve, not the sign of its minimum.

The point of shipping the reference is that agents on this task are scored on regret rather than raw PnL. A market-making leaderboard ranked on PnL rewards whichever policy drew the friendliest flow; regret against a fixed analytical reference on the same episodes removes that luck axis.

8.3 F3: Generalization gap and cross-regime transfer

`generalization_gap` scores the default reference policy (equal-weight long) on disjoint train and held-out seed bands (16 seeds each, separated by a 10,000-seed gap) within each tier. `cross_regime_transfer` then holds sixteen seeds fixed and varies the regime, producing the full three-by-three transfer matrix whose diagonal is zero by construction, because identical regimes reuse byte-identical environments. The policy is fixed and parameter-free, so nothing is trained: "source regime" below means the regime the policy is nominally selected on, and for such a policy the matrix reduces to pairwise differences of three per-tier deflated Sharpes. What F3 measures is the instrument, how much score a regime switch moves for a fixed reference, and not transfer learning. These are the 16-seed subsets Section 6.1 exempts fixed policies from replacing with the

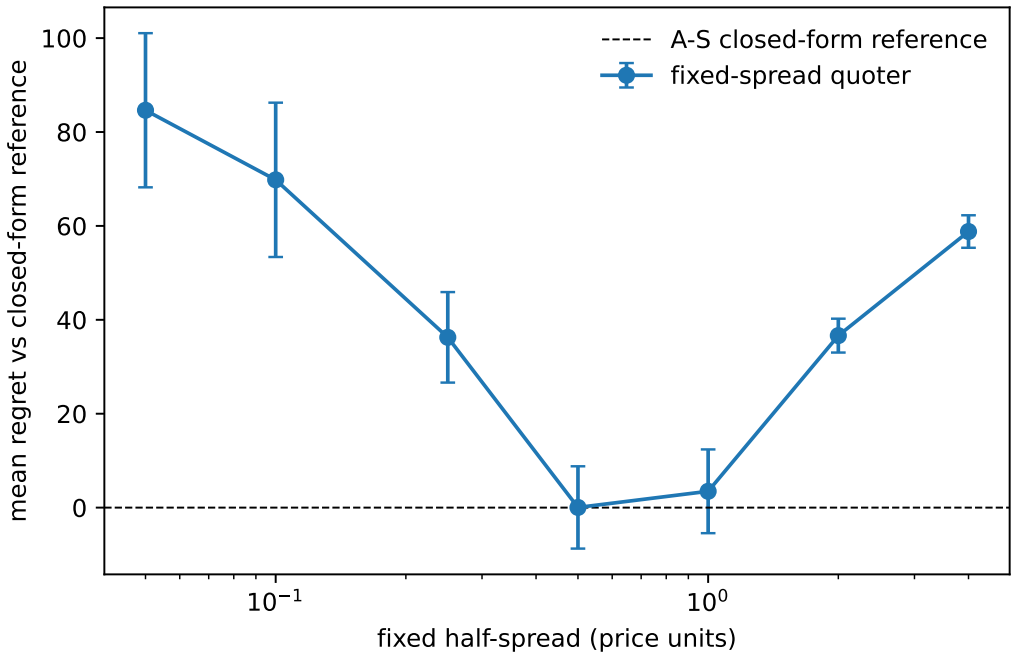


Figure 13: Mean regret versus the analytical Avellaneda–Stoikov reference policy over 16 paired episodes, with t-based 95% CIs. Fixed-spread quoting is U-shaped in the half-spread; the reference defines zero.

Table 11: Left: within-tier generalization gap (train minus test deflated Sharpe) for the fixed equal-weight reference. Right: cross-regime transfer gap (in-distribution minus zero-shot out-of-distribution DSR), rows source regime, columns scored regime; the diagonal is zero by construction. Cells are read only through their seed-paired bootstrap intervals, and the Hard↔Extreme cells (± 0.096), whose interval straddles zero, are not interpreted.

Tier	Train DSR	Test DSR	Gap
Calm	1.0000	0.9988	$+0.0012$
Hard	0.1231	0.4618	-0.3388
Extreme	0.0269	0.1280	-0.1012

	Calm	Hard	Extreme
Calm	0.000	$+0.877$	$+0.973$
Hard	-0.877	0.000	$+0.096$
Extreme	-0.973	-0.096	0.000

full canonical band. The evidence commits per-seed return series per cell; CIs are seed-resampled percentile bootstrap ($B = 2000$), seed-paired across regimes for the matrix cells.

Table 11 reports both. The within-tier gaps are the expected control: the reference policy has no fitted parameters, so it cannot overfit its train band, and the instrument correctly reads near zero on Calm ($+0.0012$) and noise-signed values on the stressed tiers (the negative Hard and Extreme gaps mean the held-out band happened to be kinder; with no learning, the gap is pure seed noise, and how much band luck a 16-seed evaluation carries is read from the interval widths rather than from any single realized gap). The per-band bootstrap intervals say the same thing at interval width: Hard's train-band DSR of 0.1231 carries a CI of $[0.000, 0.944]$ and its test band $[0.037, 0.948]$ (the bands are disjoint seed sets, so these are unpaired), overlapping almost entirely, which is what a pure-noise gap looks like. Hard's train-band DSR of 0.1231 here and Table 9's 0.1114 for the same policy on the same sixteen seeds differ only in the declared trial count the deflation prior consumes, six declared policies in F1 against no declared search in the gap metric; rescoring the committed F3 return series at six trials reproduces 0.1114 exactly.

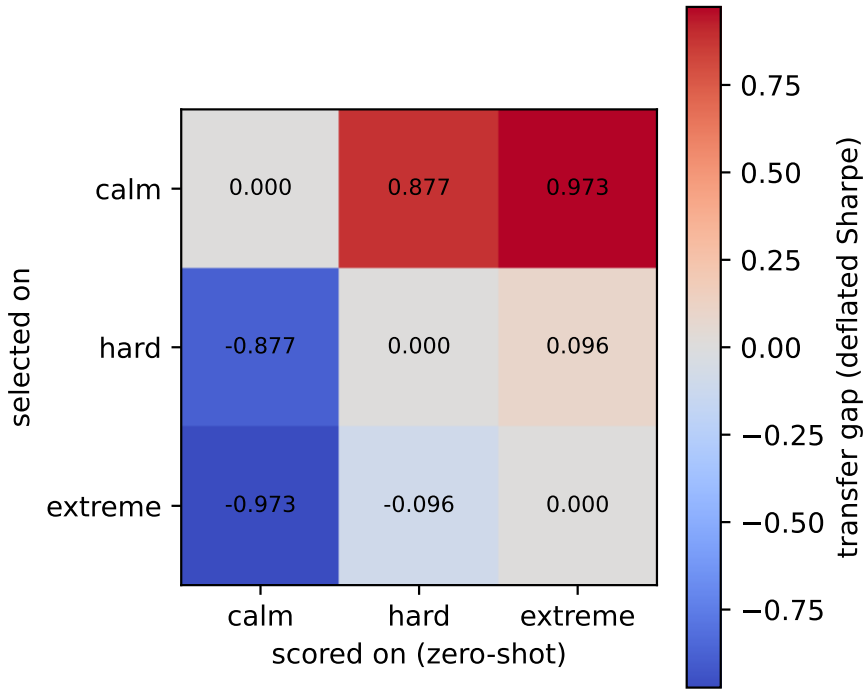


Figure 14: Cross-regime transfer matrix for the reference policy: out-of-distribution deflated Sharpe with the seed band held fixed.

The transfer matrix (Figure 14) is where regime structure appears, and every cell is classified by one rule: whether its seed-paired bootstrap interval excludes zero. The Calm-involved point estimates are the largest: source regime Calm scored zero-shot on Hard costs 0.877 of deflated Sharpe, and Calm to Extreme costs 0.973 of a score bounded in $[0, 1]$. The seed-paired bootstrap intervals temper the two claims differently: Calm to Extreme excludes zero ($[0.635, 0.999]$), while Calm to Hard's interval is wide ($[0.054, 0.999]$), positive throughout but reaching close to zero, so at 16 seeds only the Calm-to-Extreme collapse is resolved. The Hard-to-Extreme cell (0.096, CI $[-0.218, 0.911]$) has an interval straddling zero, so it is not interpreted: at 16 seeds the instrument cannot distinguish it from band luck. A bounded, heavily deflated score bootstrapped over 16 seeds is a coarse instrument, which is itself a calibration result, and Section 6.2 turns it into a prescription. The matrix is antisymmetric because the policy is fixed, and its diagonal is zero by construction. Its off-diagonal magnitude is the quantity a within-tier gap is structurally blind to: a policy scored only on calm seeds can carry a saturated DSR into a leaderboard while being worth 0.03 under stress. Both metrics are cheap, and the protocol requires reporting both.

8.4 F7: Failure-mode distributions

`classify_episode_failure` classifies every rollout of the eight-policy field (the six reference policies plus the two behavioral counterparties), across all three tiers and sixteen seeds (384 episodes), into the worst-case-first taxonomy: cascade-wiped, bankrupt, stopped-out, mandate breach (structural, drawdown or inventory), or clean, with a per-scenario sampled mandate and a 50% drawdown stop in force.

Mandates are sampled per scenario and assigned to fixed policies that cannot observe or adapt to them, so a structural breach is guaranteed whenever the draw pairs, say, a shorting policy with a long-only mandate. The flat structural-breach counts across tiers (36, 36, 31) are the signature of that

Table 12: Failure-mode counts per tier, 128 episodes each (8 policies × 16 seeds). No episode reaches bankruptcy or a cascade wipe; failure is dominated by mandate breaches, and drawdown breaches triple from Calm to Extreme.

Tier	Clean	Stopped out	Mandate struct.	Mandate DD	Mandate inv.	Bankrupt	Clean rate
Calm	72	0	36	11	9	0	0.56
Hard	68	0	36	14	10	0	0.53
Extreme	50	5	31	32	10	0	0.39

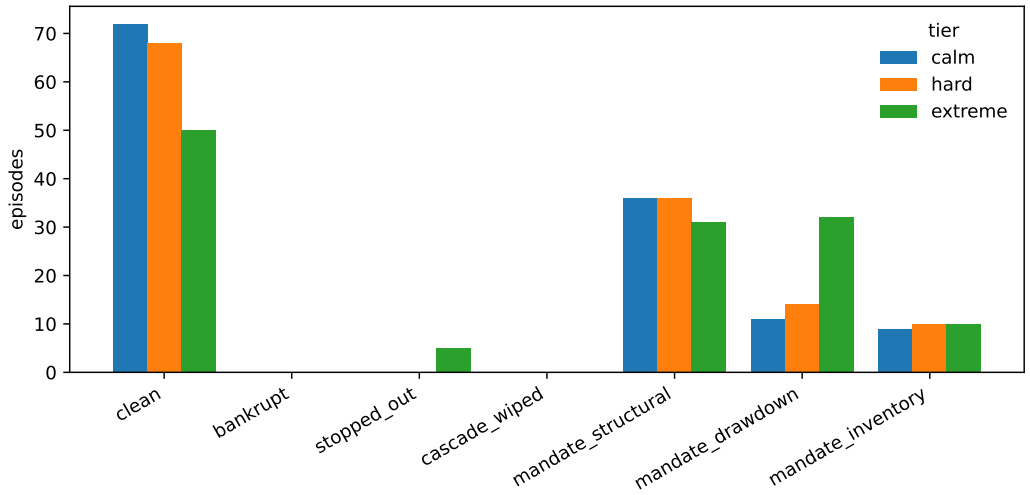


Figure 15: Failure-mode distribution per tier per policy over 384 classified episodes.

assignment mechanism, not a behavioral finding about agents; a mandate-aware field could invert the distribution entirely, and measuring one is the interesting follow-up.

Table 12 and Figure 15 show the shape of failure as stress rises. The clean rate falls from 0.56 on Calm to 0.39 on Extreme. Each rate is 128 episodes, so a difference between two tiers carries a nominal binomial 95% half-width of about 0.12; the episodes sharing a seed or a policy are not independent, so that width is a floor. The Calm-to-Hard step (0.56 versus 0.53) sits inside it and is not interpreted; the drop to Extreme is the only step that clears it. Structural mandate breaches are roughly flat across tiers, as the mechanism above dictates: a shorting policy under a sampled long-only mandate breaks the rule on every tier. The stress-sensitive modes are the drawdown family: mandate drawdown breaches triple from 11 on Calm to 32 on Extreme, and hard stop-outs at the 50% line appear only on Extreme (5 episodes, four of them `max_sharpe`). No policy in this field ever reaches bankruptcy or a cascade wipe; the taxonomy's worst states are reachable but not by these baselines at this stop level. Per policy, `flat` is clean 48/48 by construction, while `overconfident` accounts for the most Calm failures (7 of its 16 episodes end in a drawdown breach) and `max_sharpe` is the worst on Extreme (3 clean of 16). On this mandate-blind reference field, roughly half of the failures are process failures that a return-ranked board does not record.

8.5 F8: Ecology outcomes under shocks, replicated over seeds

`run_ecology` seeds the replicator with the eight baseline species over a shared-book market via `market_payoffs` (12 generations, field size 8, innovation every 4 generations breeding parameter-perturbed variants of the leader) and runs trajectories under a steady all-Calm control schedule and a `regime_shocks` schedule rotating Calm (generations 0 to 3), Hard (4 to 7) and Extreme (8 to 11). We replicate the probe over eight replicator seeds per schedule; Table 13 reports the cross-seed winner distribution, and the seed-0 trajectories (Figure 16) remain the illustrative detail. The two

Table 13: Replicated ecology outcomes over 8 replicator seeds per schedule: distribution of the final dominant lineage (bred variants credited to their founding species). The root winner differs between the schedules on 5 of 8 seeds, and on 3 of the 6 seeds where both schedules resolve a dominant lineage; the seed-0 replacement of the volatility targeter by the unlevered long book occurs on 1 of 8 seeds.

Final dominant lineage	Control (all Calm)	Shocked (Calm/Hard/Extreme)
kelly_vol_target	4	7
equal_weight_long	2	1
flat (no resolved dominant)	2	0

schedules chain episode seeds differently (the shock schedule strides seeds across generations), so control and shocked runs at the same replicator seed are matched in configuration, not in price paths.

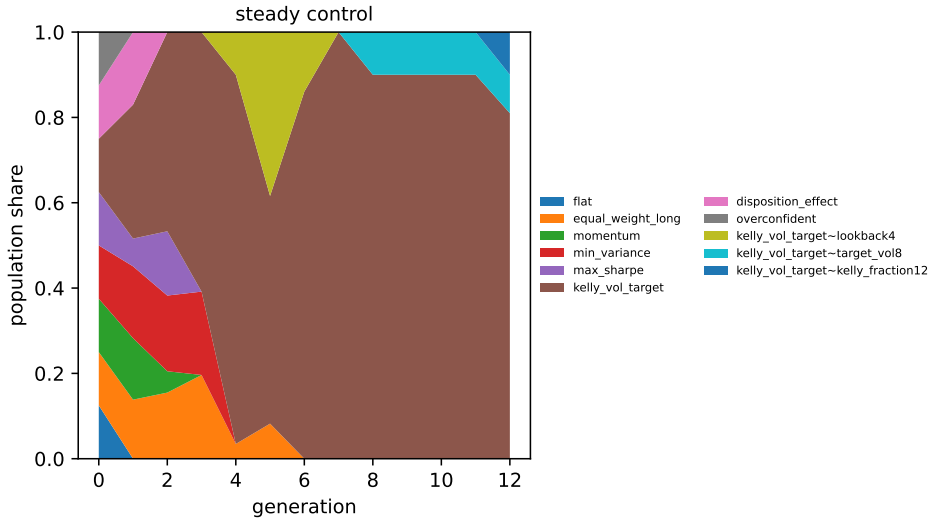
The replicator flow rule sits in the market-ecology tradition: strategies as species whose profitability decays as their share of the flow grows [Farmer, 2002], and populations whose composition, rather than any single agent's optimization, drives aggregate dynamics [Lux and Marchesi, 1999]. The implementation abstracts away much of what those models study: capital flows react instantly (no fund-flow lags), there are no leverage or margin constraints beyond the strategies' own sizing, and there is no entry beyond the single leader-perturbation innovator. Its outputs are trajectories of this replicator map, not claims about market ecology in general.

The replication does not support the single-seed narrative. At seed 0 the story was clean: `kelly_vol_target` sweeps the control board (peak share 1.00, final 0.81), collapses under the shock schedule once the regime turns Hard, and `equal_weight_long` inherits dominance (final 0.67). Across eight seeds that story is the exception. Under the shock schedule a `kelly_vol_target` lineage ends dominant on 7 of 8 seeds (final shares near 0.90); the seed-0 handoff to the long book happens once. The control side is itself seed-dependent: `kelly_vol_target` wins 4 seeds, `equal_weight_long` wins 2, and on 2 seeds no species resolves dominance at all (all eleven end persistent at small shares, with a `flat` variant holding the largest share at 0.10). The root-level winner differs between the two schedules on 5 of 8 seeds, but on 2 of those the control side never resolves a dominant lineage at all; among the 6 seeds where both schedules resolve one, the winner differs on 3. At eight seeds the reordering is neither established nor excluded, and it has no consistent direction. Extinction pressure is more stable than identity: shocked runs produce a resolved dominant on 8/8 seeds with 4 to 7 extinctions per run (at seed 0, both behavioral counterparties are gone or collapsed on both schedules, `overconfident` extinct at generation 0).

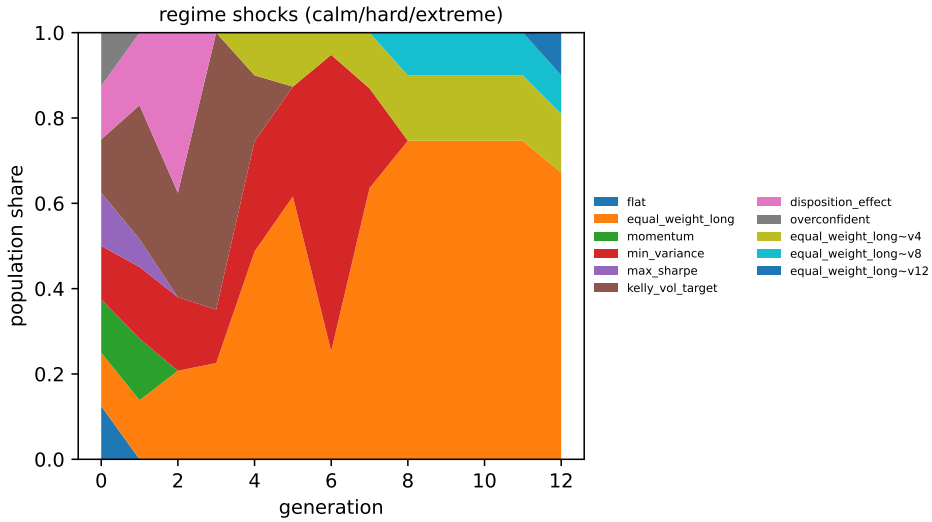
One replicator trajectory is one draw from a heavy-tailed outcome distribution, and single-run ecology narratives do not survive replication. What the probe reliably produces at this scale is the outcome distribution itself: which lineages can win, how often shocks reorder the board, and how much of the population goes extinct, per committed seed set. Sharper claims need more seeds, varied shock orderings and path-matched schedule pairs, all of which are mechanical extensions of the committed script.

9 Related work

Environment interfaces. Gym [Brockman et al., 2016] demonstrated that a small, stable environment interface is worth more than any single environment, and Gymnasium [Towers et al., 2024] carried the interface forward with versioned environment IDs and a conformance checker; SharpeArena's Python surface is a first-class Gymnasium citizen with versioned, namespaced IDs and adopts its `check_env`. PettingZoo [Terry et al., 2021] standardized the multi-agent case, which SharpeArena's shared-book and batched-competition environments implement. Minari [Farama Foundation, 2024] standardizes offline-RL datasets; SharpeArena exports leak-safe rollouts to it. The wire contract of Section 4 differs from all of these in scope: it is a language-agnostic JSON boundary for external agent processes, governed like a protocol rather than a library API.



(a) Control: steady Calm.



(b) Shocked: Calm, then Hard, then Extreme.

Figure 16: Seed-0 population-share trajectories over 12 replicator generations, the illustrative single run behind Table 13's distribution. At this seed the regime rotation replaces the calm-regime winner with the unlevered long book; across the eight-seed replication that replacement occurs once.

Table 14: Financial RL environments on the five property axes of Section 1. Entries describe the mechanism each system provides for the axis, not the quality of the system on its own terms. Where an entry rests on our reading of a system's paper and repository rather than on a property its authors claim, it is written descriptively and marked "not targeted" rather than "absent".

System	Leak-freedom mechanism	Determinism guarantee	Trajectory verifiability	External-agent contract	Generalization protocol
FinRL [Liu et al., 2020]	pipeline discipline over replayed history	not targeted	not targeted	in-process Gym API	train/test date split
FinRL-Meta [Liu et al., 2022]	DataOps pipeline discipline	not targeted	not targeted	in-process Gym API	train/test date split
ABIDES-Gym [Amrouni et al., 2021]	discrete-event simulation kernel	not targeted	not targeted	in-process Gym API	not targeted
TradeMaster [Sun et al., 2023]	workflow convention	not targeted	evaluation toolkit inside the agent's trust domain	in-process Python API	train/valid/test split
mbt_gym [Jerome et al., 2023]	model-generated state, no history to leak	not targeted	not targeted	in-process Gym API	not targeted
ABIDES [Byrd et al., 2020]	discrete-event simulation kernel	not targeted	not targeted	in-process agent classes	not targeted
SharpeArena	interface shape: no operation returns a post-cursor bar (Section 3.1)	committed FNV-1a goldens across native, WASM and Python (Section 3.3)	recompute-from-decisions replay with an adversarial tamper test (Section 3.4)	governed JSON wire contract v1.0 with schemas and fixtures (Section 4)	disjoint integer seed bands with a 10,000-seed gap (Section 6.1)

Evaluation rigor in RL. The environment's evaluation stance imports three lessons from the RL evaluation literature. From the ALE revisitation [Machado et al., 2018]: determinism in the environment invites trajectory memorization, so evaluation must be designed against it; SharpeArena keeps determinism for verifiability and defeats memorization with procedural scenario distributions and held-out seed bands instead of injected stochasticity, and Section 7.4 measures how far that gets. From Procgen [Cobbe et al., 2020]: procedural generation over integer seed intervals makes generalization measurable, which Section 3.5 ports to markets and Section 6.1 states as a protocol. Prioritized Level Replay [Jiang et al., 2021] shows seed-level curricula shape what agents learn, which motivates keeping the train band operator-owned and the held-out band frozen. τ-bench [Yao et al., 2024] argues that agent reliability must be measured across repeated runs rather than on average; that requirement lives in the SharpeBench kernel this environment feeds [Toca, 2026].

Financial RL environments. A Gym-for-finance lineage predates this work. Table 14 places SharpeArena against it on the five property axes the environment competes on, which are the axes Section 1 names and the rest of this paper enforces.

The table compresses five substantive differences. FinRL and FinRL-Meta wrap historical market data into gym-style training environments at scale, and FinRL-Meta's DataOps pipeline explicitly targets the data-quality half of the problem, survivorship bias and backtest overfitting included; because their environments replay recorded data inside a general-purpose Python process, leak-freedom is a property of pipeline discipline rather than of interface shape, and a caller who reaches around the wrapper is not stopped by anything. ABIDES-Gym wraps the ABIDES discrete-event market simulator in Gym environments, bringing multi-agent market microstructure to RL training; its focus is simulation fidelity, and to our knowledge it does not target cross-runtime byte-determinism or tamper-evident trajectories. TradeMaster is a holistic platform spanning data, environments, algorithms and evaluation for RL trading research; it standardizes the workflow rather than hardening the producer, and its evaluation toolkit runs inside the same trust domain as the agent, which is the arrangement Section 6.4 declines. mbt_gym provides model-based Gym environments for limit-order-book problems, including the same Avellaneda–Stoikov family as Section 3.6, with closed-form solutions available as baselines; it is the closest relative of the market-making task specifically, and Section 3.6 notes the relation. Where these descriptions rest on our reading of the systems' papers and repositories rather than on properties their authors claim, we have kept them descriptive; none of these systems, to our knowledge, sets out to make look-ahead structurally unavailable at the interface,

pin cross-runtime byte-identity with committed hashes, or make submitted trajectories third-party recomputable, which is the specific combination this paper contributes.

Agents across a process boundary. The external-agent column of Table 14 reads "in-process Gym API" or "in-process agent classes" for every prior financial environment, and that entry decides more than it appears to. An in-process agent is a Python object; calling it either returns a decision or raises, and there is no third outcome to type. A wire contract admits one, because a transport can fail without either party being wrong, and the failure then has to be represented somewhere or it will be absorbed by whatever the transport returns instead. The failure mode of Section 4.4 therefore does not exist for these systems, and its absence from their designs is not an oversight: they do not have the capability that creates it. The agent-evaluation literature outside finance does have the capability, and τ-bench [Yao et al., 2024] is the closest neighbour on the measurement question, arguing that agent reliability is a property of repeated runs rather than of an average and building the pass^k statistic to say so. Its reliability gate lives in the SharpeBench kernel this environment feeds [Toca, 2026]. What SharpeArena adds on this axis is upstream of reliability rather than a competitor to it: a run that recorded a wire fault is not a low-reliability run, it is not a run, and the distinction has to be made before any statistic is computed over the k repetitions.

Market simulation and its validation. ABIDES [Byrd et al., 2020] is the closest simulator in spirit: a high-fidelity multi-agent market with a matching engine and background agent populations. SharpeArena trades some of that fidelity for properties ABIDES does not target, listed in Table 14. The microstructure models are the classical ones: Kyle's permanent impact [Kyle, 1985], Almgren–Chriss temporary impact and execution cost [Almgren and Chriss, 2001], and the Avellaneda–Stoikov analytical quoting reference [Avellaneda and Stoikov, 2008]. The realism diagnostic compares the generator with selected stylized facts from Cont [2001], and Section 7 argues from the validation literature [Platt and Gebbie, 2018, Fagiolo et al., 2019, Lamperti, 2018, Vyetrenko et al., 2020, Nagy et al., 2025] that a calibrated gate a public generator can fail is worth more than an uncalibrated one it passes. That literature is also where SharpeArena's gate stops: it is a directional three-check conjunction, not a distributional divergence in the sense of Lamperti [2018] or a full L1-style benchmark suite in the sense of Nagy et al. [2025], and adopting one of those is the natural next revision of the diagnostic.

Scoring. The environment deliberately contains no scorer. The deflated Sharpe [Bailey and de Prado, 2014] and the reliability and process gates that consume this environment's trajectories are specified in the SharpeBench paper [Toca, 2026]; this paper is the producer half of that funnel.

10 Limitations

One synthetic generator family. All three tiers come from a single procedural vol-and-jump generator; no real price series enters the environment, and the tiers differ in that generator's parameters, not in kind. F4 quantifies the cost: the tape has fat tails on the stressed tiers but weak volatility clustering at episode length, and the finite-panel-calibrated three-check conjunction passes on only 1 of 24 seeded panels. The Fano ratio is conditional-IID-calibrated but exploratory and does not gate. The existing Calm calibration grid produces no preset that meets its diagnostic, confirmation and volatility rule. An agent that excels here has demonstrated skill against this generator's structure, not against a market, and the realism gate says so rather than hiding it. Section 7 argues that a gate a committed public generator can fail is evidence the gate discriminates, which is a claim about the instrument and not a defence of the tape: on the four-level empirical-validity ladder of Fagiolo et al. [2019] the canonical generator sits at Level 0 on the gated axis, and every finding built on it is instrument calibration on uncertified tape. The paragraph below states which experiments run on other processes instead.

The interface guarantee is not an information guarantee. The leak-freedom of Section 3.1 is a statement about the interface's shape: no operation returns a post-cursor bar. It is not a claim

that public deterministic scenarios are unpredictable. Section 7.4 makes the distinction empirical. A causal AR(5) adversary stays near its unconditional baseline, but an exhaustive scan of a public 2^{16} seed band recovers all 16 tested seeds from one observed bar in about one second. Full-space brute force remains infeasible. A complete 64-bit SplitMix64 output inverts to one state, whereas its published 53-bit unit representation leaves 2^{11} candidates; tape-level inversion through the later price arithmetic remains open. The opt-in sealed derivation of Section 7.4.1 defeats this specific bounded-band scanner 0/16 when its salt is withheld, and a forward protocol can commit the salt hash before entry. The evidence here is a simulated workflow, not evidence that any live evaluation used it. Any public bounded-band evaluation should be treated as compromised by this scanner until it documents sealed seed custody, a pre-entry commitment, non-reuse and a reveal deadline. The derivation is PRF-style, not a certified MAC, so its security rests on salt entropy and secrecy rather than a cryptographic proof.

The baselines are trivial policies, not trained agents. Every number in Sections 7 and 8 is produced by fixed reference policies (buy-and-hold analogs, one-step tilts, closed-form allocators) and two deliberately biased behavioral counterparties. They fix the zero point of each instrument and calibrate its resolution; they do not establish what a trained agent scores, and this paper's claims are scoped to environment and protocol for that reason. The constraint is the learner's, not the simulator's: Section 6.6 measures the environment at 14,240 steps per second on the slowest of the three shipped surfaces, so a PPO run is bounded by the policy's own training cost rather than by environment time. In particular, the near-zero within-tier generalization gaps in F3 are the expected control for policies with nothing to overfit, not evidence that the environment resists overfitting by learners. Relatedly, no reference policy is rank-eligible under the kernel's full conjunction (Section 8.1). Section 8.1.1 establishes that the eligibility set is non-empty on every tier, but it does so with an out-of-band oracle that replays the seed one bar ahead, which is not an entrant and is not scored on any board; no policy that reads only the observation interface has yet been shown eligible.

Named future experiments. Two follow-ups remain specific. (1) A learning baseline: PPO trained through the shipped Gymnasium adapter on the train band, evaluated on a named disjoint evaluation band and cross-regime, so F3 contains a policy that can actually overfit. (2) Tape-level analytic state recovery from quantized SplitMix64-derived prices. The exact-finalizer inverse and the resulting 2^{11} candidates per 53-bit unit are tested, but no observed price has been inverted; sealed seeds (Section 7.4.1) defeat this specific bounded-band scanner but do not answer this one. The manipulation positive control in Section 7.2.2 finds a profitable sampled round trip at $\beta = 0.5$ and none among its sampled linear-impact cells, though the search did not cover all interactions or transient impact. The rank-eligibility existence question of Section 8.1 is answered by the witness of Section 8.1.1, by an oracle rather than by an entrant.

The Python probe layer is outside the golden-hash guarantee. The byte-identical claim covers the Rust core, the WebAssembly build and the generated scenario bytes returned through the Python binding: the native and Python test suites assert all seven committed golden fingerprints and the WebAssembly suite asserts the canonical and clustered ones, and direct native–WebAssembly parity tests enforce the second. The Python binding delegates environment dynamics to that same native Rust core and has deterministic replay tests, but the golden coverage on the Python side is the scenario serialization, not the package's other serialized artifacts. The Python reductions that produce the realism, manipulation, adverse-selection, failure, ecology and predictability evidence are deterministic in their seeds, but are off the golden-hash path; their floating-point reproducibility is the platform's, not the paper's. The concave-impact engine itself is also excluded from the canonical byte-identity claim because a general exponent uses `powf`; the linear default remains on the untouched golden path.

The realism diagnostic does not certify every experiment's process. F4 applies to the canonical position-trading generator. The Avellaneda–Stoikov market-making process in F2 and the market-clearing models in F5 and F6 are separate simulators with their own structural and positive-control

checks; they are not passed through F4. Their results are therefore calibrations of those specified models, not claims conditioned on the canonical tape's realism verdict. The distinction matters in both directions: F4's failure does not mechanically invalidate a different process, and the separate process is not market-valid merely because its internal controls pass.

The impact models are stylized. The endogenous market composes Kyle and Almgren–Chriss terms; the adverse-selection module does not separate a meta-order's information from its impact; the follower population is a one-parameter caricature. The concave ablation is not a universal test: its dimensionless flow $|Q/V|$ sits two to three orders of magnitude below the unit crossover, so exponents below one amplify impact relative to the linear arm at the tested calibration, and the symmetric schedule does not search the asymmetric round trips nonlinear theory permits. Section 7.2.2 does search them and finds cells that pay at $\beta = 0.5$ with no followers, so the symmetric null says this pump does not pay here, and the positive control says the instrument can see schedules that do; the extended sweep enlarges the sampled axes but does not cover interactions, other flow scales or transient impact.

The guard is a deny-list. The structural guarantee covers the environment's own surface. `LookaheadGuard` extends it to harness tools by pattern, and a deny-list is exactly as good as its patterns; a novel leak path through a side channel the environment does not own (wall-clock time, filesystem state, a network call) is out of scope for both layers. The replay check verifies decisions against frozen data and therefore catches falsified results, not information an agent obtained elsewhere.

The local-agent field is built, and still unrun. The contract, conformance kit, replay verifier, strict shared parser, transport-fault gate, edge manifest and its ledger, counterfactual ledger, batched local-model scheduler, resumable evidence journal, trace-promotion path, paper-execution lifecycle and SharpeBench artifact compiler exist and are tested with deterministic model doubles. They do not constitute agent evidence. No open-weight model has completed the predeclared field, no third party has submitted an agent, and no performance number from the local path appears in this paper. The frontier-model inventory also exceeds the workstation: current trillion-parameter sparse releases are relevant comparison targets but cannot load on its 16 GB GPU and 256 GB RAM, so a first local field necessarily tests smaller exact checkpoints and labels any partial-offload calibration arm separately. Model weights may also contain public historical prices; local execution closes provider drift and network leakage, not pretraining contamination. A hosted intake, live leaderboard and multi-tenant service do not exist. The significance claim remains the verifiability infrastructure that is checkable today; adoption and model performance are future evidence rather than assumptions.

Containment is the companion crate's, and no test has yet observed it hold. Two different claims travel under the word sandbox, and only one of them is this artifact's. The leak-freedom of Section 3.1 and the `LookaheadGuard` behind it are properties of code in this repository, asserted by tests that run here. Hostile-code containment is not: the fail-closed Docker launcher that would isolate an executable third-party entrant lives in the companion `sharpebench-arena` crate of the SharpeBench repository, and this crate contains no containment code at all. That launcher configures network and IPC isolation, a read-only root, non-root execution, dropped capabilities, no-new-privileges, seccomp and explicit CPU, memory, process and file-descriptor limits. Those flags describe an intended boundary. They are not a penetration test, and not a proof against a kernel, Docker or GPU-runtime escape.

The boundary has never been observed to hold, and the reason is worth stating rather than summarizing. Its smoke test guarded itself with a runtime check that returned green whenever Docker was absent, and a skip that returns green is indistinguishable from a pass in the test summary, so the suite reported no ignored tests for the whole life of the test while the container boundary went unexercised. The assertion the test did carry would have been satisfied by any failure mode, a container that never started included. Both container tests now carry `#[ignore]` with the daemon and fixture they need named in the attribute, and each opens by asserting that Docker is available, so an absent daemon

under an explicit `-ignored` run is a failure rather than a silent pass; ignored tests are counted and named, so an operator can now see that the boundary was not exercised. Seeing it is not verifying it. The development machine's Docker daemon is not running, so neither test has executed, and the negative fixtures for egress, host-file access, fork exhaustion and cleanup remain acceptance work before any untrusted executable field. Until one of them runs on a Docker-enabled host, nothing in this paper should be read as evidence that hostile code is contained. The generated-strategy path of Section 5.3 avoids the exposure entirely by accepting only a closed, non-executable language, and the local model server stays outside the container as part of the trusted computing base.

Partial fills are last-write-wins. The order lifecycle's partial-fill transition sets the cumulative filled quantity from the value the caller supplies rather than summing deltas, so a sequence of partial reports leaves the last one standing. That is what the paper-broker adapters shipped here require, because each report carries a cumulative reading, and the self-transition exists precisely so successive readings are legal. A broker that instead sends incremental deltas would accumulate wrongly against this code and needs different handling; the interface does not currently distinguish the two conventions, and adopting such a broker is a change to the lifecycle rather than a configuration choice.

Reconciliation does not retry, so a flaky broker halts a session. The sweep over unresolved orders is a single pass with no retry and no backoff. An unanswerable query raises, and the raise propagates out of the sweep, so orders it had not yet reached are left for a later call and the session stops. This is the safe direction and it is deliberate: the alternative, swallowing the failure and continuing, is exactly how an unanswerable query becomes a recorded absence and a recorded absence authorizes a duplicate order. The cost is a real one. A broker that answers intermittently will halt a forward session that a retry with backoff would have completed, and the operator has to restart the sweep by hand.

Forward evidence is not historical replay. The paper-trading arm is deliberately paper-only, places no real-capital order and gives the model no credential. Its trusted supervisor applies a fixed risk gate and records every decision, refusal, submission and broker response. Even so, provider-time-dependent bars and simulated fills cannot be reproduced byte for byte after the fact. A paper result must therefore be labeled as forward, non-replayable evidence and bound by commitment, raw-response hashes and broker acknowledgments; it cannot be pooled silently with the deterministic historical tables in this paper, and the admissibility rule of Section 6.5 is what keeps the two apart. A forward record can also be incomplete and still honest: an order left in extttsubmission_unknown by an unanswered query (Section 5.6) is a fact about what the run observed, and resolving it to accepted or absent without asking the broker would be the only way to make the record look complete.

Broader impact. Two dual-use surfaces deserve an explicit norm. First, leaderboard scores: a byte-verifiable score is more persuasive, not more externally valid, and a verified SharpeArena number is a claim about a synthetic environment only. SharpeArena scores are not performance marketing, and presenting them to retail audiences as evidence of real-market skill would be a misuse the project's documentation states as out of bounds. Second, the manipulation probe: it is a payoff-search harness over manipulation schedules, which is dual-use by construction. We judge publishing it net-positive because the linear-impact arm is a diagnostic against the precise no-manipulation assumptions stated in Section 7.2, not a search for deployable trades; all 45 sampled linear configurations lose money, but the paper makes no guarantee outside that finite grid. The module carries its simulator-diagnostic disclaimer in code, and the crude, legible schedules it implements are the ones surveillance already catches in real markets, so the probe teaches defenders more than attackers. The profitable concave cells use sliced accumulation and block liquidation, a schedule family motivated by the published manipulation literature rather than a claim about an executable market strategy.

11 Reproducibility statement

The environment is a Rust workspace of three crates under the MIT and Apache-2.0 licenses, published to crates.io, npm and PyPI, with a pinned toolchain and a lockfile, depending on the published SharpeBench engine crates rather than a vendored copy. The core forbids unsafe code. The determinism claims are enforced rather than asserted: committed FNV-1a golden hashes pin the generated scenarios and the matching engine's fill tape, the WebAssembly crate's tests assert byte-identity with the native engine for both stepping and scenario generation, and the npm package's smoke test performs the replay-and-tamper check of Section 3.4 on every run. The contract artifacts, schemas, conformance fixtures and the governance document, are committed beside the code they govern, and a conformance test exercises the fixtures. Continuous integration covers the Rust surface with formatting, lints as errors, tests and a WASM target build, plus dependency auditing, the npm package and the Python wheel.

Every number in Sections 7 and 8 is produced by a committed script under `paper/src/` from fixed seeds, writing JSON evidence to `paper/evidence/` and figures to `paper/figures/`; the commands are listed in Section A. The scripts call the package's public Python API. Reproduction therefore requires the source revision recorded by the evidence manifest, its locked dependencies and the listed commands; a package version alone is not a complete environment specification.

The local-agent, generated-strategy and paper-only forward paths of Section 5 are product surfaces, not sources for any number in this manuscript. Nothing has been promoted through the path of Section 6.5 into the committed evidence set, so every artifact under `paper/evidence/` is producer output rather than a promoted trace; when that changes, the promotion record is what will distinguish the two inside the same manifest. Their deterministic tests use injected model transports and fixed decisions; continuous-batched inference, model-server scheduling and live provider data are outside the golden-hash claim. A future model field must record the exact checkpoint revision and digest, license, quantization and converter, inference engine and version, chat template, constrained-decoding backend, GPU/offload map, context and batch settings, sampling seed, thinking budget, raw completion, semantic-validation outcome and fault class. Environment bytes and replay can remain fixed while agent outputs vary, and the evidence schema preserves that distinction.

The products compose through files rather than a mutual dependency. SharpeArena writes and validates complete append-only cell journals, then compiles one SharpeBench submission artifact per dataset; SharpeBench reads those artifacts and remains the independent field-level evaluator. A Gordon architecture review informed the threat model and a future scaffold ablation, but no Gordon module, runtime or broker adapter is imported into the reported environment or the built minimal local scaffold.

No large language model is a component of the environment, the contract or any reported result. The environment has no judge. We used language models to draft and edit prose, and we checked every bibliography entry against its source.

References

Franklin Allen and Douglas Gale. Stock-price manipulation. *The Review of Financial Studies*, 5(3): 503–529, 1992.

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3(2): 5–39, 2001.

Robert Almgren, Chee Thum, Emmanuel Hauptmann, and Hong Li. Direct estimation of equity market impact. *Risk*, 18(7):58–62, 2005.

Selim Amrouni, Aymeric Moulin, Jared Vann, Svitlana Vyetrenko, Tucker Balch, and Manuela Veloso. ABIDES-Gym: Gym environments for multi-agent discrete event simulation and application to financial markets. In *Proceedings of the Second ACM International Conference on AI in Finance (ICAIF)*, 2021.

Marco Avellaneda and Sasha Stoikov. High-frequency trading in a limit order book. *Quantitative Finance*, 8(3):217–224, 2008.

David H. Bailey and Marcos López de Prado. The deflated sharpe ratio: Correcting for selection bias, backtest overfitting, and non-normality. *The Journal of Portfolio Management*, 40(5):94–107, 2014.

Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. OpenAI Gym, 2016.

Eric Budish, Peter Cramton, and John Shim. The high-frequency trading arms race: Frequent batch auctions as a market design response. *The Quarterly Journal of Economics*, 130(4):1547–1621, 2015.

David Byrd, Maria Hybinette, and Tucker Hybinette Balch. ABIDES: Towards high-fidelity multi-agent market simulation. In *Proceedings of the 2020 ACM SIGSIM Conference on Principles of Advanced Discrete Simulation (SIGSIM-PADS)*, 2020.

Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning*, 2020.

Rama Cont. Empirical properties of asset returns: Stylized facts and statistical issues. *Quantitative Finance*, 1(2):223–236, 2001.

Giorgio Fagiolo, Mattia Guerini, Francesco Lamperti, Alessio Moneta, and Andrea Roventini. Validation of agent-based models in economics and finance. In Claus Beisbart and Nicole J. Saam, editors, *Computer Simulation Validation: Fundamental Concepts, Methodological Frameworks, and Philosophical Perspectives*, Simulation Foundations, Methods and Applications, pages 763–787. Springer International Publishing, Cham, 2019. doi: 10.1007/978-3-319-70766-2_31. Circulated as LEM Working Paper 2017/23, Sant'Anna School of Advanced Studies.

Farama Foundation. Minari: A dataset API for offline reinforcement learning. `https://minari.farama.org`, 2024.

J. Doyne Farmer. Market force, ecology and evolution. *Industrial and Corporate Change*, 11(5): 895–953, 2002.

Jim Gatheral. No-dynamic-arbitrage and market impact. *Quantitative Finance*, 10(7):749–759, 2010.

Lawrence R. Glosten and Paul R. Milgrom. Bid, ask and transaction prices in a specialist market with heterogeneously informed traders. *Journal of Financial Economics*, 14(1):71–100, 1985.

Olivier Guéant, Charles-Albert Lehalle, and Joaquin Fernandez-Tapia. Dealing with the inventory risk: A solution to the market making problem. *Mathematics and Financial Economics*, 7(4): 477–507, 2013.

Gur Huberman and Werner Stanzl. Price manipulation and quasi-arbitrage. *Econometrica*, 72(4): 1247–1275, 2004.

Joseph Jerome, Leandro Sánchez-Betancourt, Rahul Savani, and Martin Herdegen. mbt-gym: Reinforcement learning for model-based limit order book trading. In *Proceedings of the Fourth ACM International Conference on AI in Finance (ICAIF)*, 2023.

Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.

Guy L. Steele Jr., Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages and Applications (OOPSLA)*, pages 453–472, 2014.

Albert S. Kyle. Continuous auctions and insider trading. *Econometrica*, 53(6):1315–1335, 1985.

Francesco Lamperti. An information theoretic criterion for empirical validation of simulation models. *Econometrics and Statistics*, 5:83–106, 2018. doi: 10.1016/j.ecosta.2017.01.006.

Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. FinRL: A deep reinforcement learning library for automated stock trading in quantitative finance, 2020. Deep RL Workshop, NeurIPS 2020.

Xiao-Yang Liu, Ziyi Xia, Jingyang Rui, Jiechao Gao, Hongyang Yang, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning. In *Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track*, 2022.

Thomas Lux and Michele Marchesi. Scaling and criticality in a stochastic multi-agent model of a financial market. *Nature*, 397:498–500, 1999.

Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

Peer Nagy, Sascha Yves Frey, Kang Li, Bidipta Sarkar, Svitlana Vyetrenko, Stefan Zohren, Ani Calinescu, and Jakob Nicolaus Foerster. LOB-Bench: Benchmarking generative AI for finance – an application to limit order book data. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*, pages 45437–45460, 2025. arXiv:2502.09172.

Donovan Platt and Tim Gebbie. Can agent-based models probe market microstructure? *Physica A: Statistical Mechanics and its Applications*, 503:1092–1106, 2018. doi: 10.1016/j.physa.2018.08.055. arXiv:1611.08510.

Shuo Sun, Molei Qin, Wentao Zhang, Haochong Xia, Chuqiao Zong, Jie Ying, Yonggang Xie, Lingxuan Zhao, Xinrun Wang, and Bo An. TradeMaster: A holistic quantitative trading platform empowered by reinforcement learning. In *Advances in Neural Information Processing Systems 36, Datasets and Benchmarks Track*, 2023.

Jordan Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. Pettingzoo: Gym for multi-agent reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Tiberiu Toca. SharpeBench: A Luck-Robust Benchmark for Trading Agents. Preprint, `https://github.com/general-liquidity/sharpebench`, 2026. Version 0.11.0.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, Rodrigo Perez-Vicente, Andrea Pierré, Sander Schulhoff, Jun Jet Tai, Hannah Tan, and Omar G. Younis. Gymnasium: A standard interface for reinforcement learning environments, 2024.

Svitlana Vyetrenko, David Byrd, Nick Petosa, Mahmoud Mahfouz, Danial Dervovic, Manuela Veloso, and Tucker Hybinette Balch. Get real: Realism metrics for robust limit order book market simulations. In *Proceedings of the First ACM International Conference on AI in Finance (ICAIF)*, pages 1–8, 2020. doi: 10.1145/3383455.3422561. arXiv:1912.04941.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains, 2024.

Gilles Zumbach. Time reversal invariance in finance. *Quantitative Finance*, 9(5):505–515, 2009.

A Commands that produce every number

All commands run from the repository root with the `sharpearena` Python package installed. Each script fixes and serializes its seeds, and the evidence manifest records file hashes and producer commands.

Version provenance. This is the paper's single provenance statement. F1 serializes `sharpearena` 0.9.0; several other evidence files do not serialize a runtime version, so this paper does not reconstruct one from their creation dates, and where present they record SharpeBench 0.5.0. F4 and F5 are generated by the documented finite-panel estimator and normalized-flow producers, and the witness artifact serializes its producer and common-random-number configuration.

Content provenance.

```
python paper/src/make-provenance.py
python paper/src/check-provenance.py
```

The first command writes `paper/evidence/provenance.json`; the second validates it against the tree. The manifest records the commit it was generated at, whether the working tree was dirty at that moment, a content hash over the Rust, Python, manifest and producer-source snapshot, and the SHA-256 digest of every JSON and figure artifact. The source snapshot is source only: paths under build directories, virtual environments and bytecode caches are excluded, so the same tree yields the same snapshot hash on a different machine. The manifest is a release-time snapshot rather than a continuously maintained view of the working tree: regenerating it is part of the release procedure, and `check-provenance.py` recomputes every recorded digest, re-expands the recorded scope and exits nonzero on any mismatch, so a manifest that has fallen behind its tree is a detectable failure rather than a silent one. Read that way, the content snapshot rather than a package label identifies the source state the numbers in this paper were produced from.

The design-property checks (Sections 3.1, 3.3, 3.4, 4, 4.4 and 5).

```
cargo test -p sharpearena              # golden hashes, leak-free
                                       # property tests, conformance kit
cargo test -p sharpearena-wasm         # native == wasm parity + golden
npm test --prefix npm/sharpearena      # replay + tamper detection
python -m pytest crates/sharpearena-py # Python surface, check_env,
                                       # seed-band disjointness, canonical
                                       # parser, local-field scheduler,
                                       # strategy ledger and paper-only arm
```

The local-field tests inject deterministic model transports and do not download weights or produce an agent-performance result. The MCP regression cases assert that unknown and duplicate symbols and action/weight contradictions fail without advancing the environment. The paper-arm tests use in-memory transports and fixed endpoints; they do not send a broker order. The transport-gate cases of Section 4.4 include the characterization test that runs a wedged agent through the unguarded engine and asserts a full-length scoreable return series, so the defect the gate closes stays visible in the suite rather than only in the changelog. The container tests of the companion extttsharpebench-arena crate are marked exttt#[ignore] and are not exercised by any command above; Section 10 states their status.

F1: baseline leaderboard (Section 8.1).

```
python paper/src/make-f1-baselines.py
  -> paper/evidence/f1-baselines.json, paper/figures/f1-baselines.pdf
```

The script passes the bootstrap parameters explicitly ($B = 2000$ percentile resamples, $\alpha = 0.05$, resample seed `0x5BA7_2026`) and serializes them in the evidence config; they equal the

library defaults defined in `crates/sharpearena-py/python/sharpearena/confidence.py` (`DEFAULT_N_BOOT`, `DEFAULT_RESAMPLE_SEED`), which reuse the scoring kernel's canonical bootstrap seed.

F2: regret vs the analytical reference (Section 8.2).

```
python paper/src/make-f2-regret.py
  -> paper/evidence/f2-regret.json, paper/figures/f2-regret.pdf
```

F3: generalization gap and cross-regime transfer (Section 8.3).

```
python paper/src/make-f3-generalization.py
  -> paper/evidence/f3-generalization.json,
     paper/figures/f3-transfer-matrix.pdf
```

F4: stylized-facts realism certification (Section 7.1).

```
python paper/src/make-f4-realism.py
  -> paper/evidence/f4-realism.json, paper/figures/f4-realism.pdf,
     paper/figures/f4-calm-calibration.pdf
```

The same command also runs the 99-configuration Calm calibration sweep, serializes it under `calm_calibration` and writes `paper/figures/f4-calm-calibration.pdf` (Section 7.1.1).

F5: manipulation boundary and size response (Section 7.2).

```
python paper/src/make-f5-manipulation.py
  -> paper/evidence/f5-manipulation.json,
     paper/figures/f5-boundaries.pdf, paper/figures/f5-size-response.pdf
```

The same command also runs the concave-impact extension at $\beta = 0.5$ and 0.7, serializes it under `concave`, and writes `paper/figures/f5-concave.pdf` (Section 7.2.1). It then runs the asymmetric positive control (Section 7.2.2): 135 grid points over five duration ratios, three block fractions and $\beta \in \{1.0, 0.7, 0.5\}$ on the theory, temporary-impact and canonical arms, eight seeds each, plus one fixed selected cell on 32 fresh seeds under `positive_control.confirmation`; the producer serializes all configurations and writes `paper/figures/f5-positive-control.pdf`. It also runs the 60-cell extended sweep using independently normalized uniform increments for unequal legs, recorded under `extended_sweeps` and written to `paper/figures/f5-extended-sweeps.pdf`. Because its anchor is selected from the first grid on the same eight seeds, each extension interval corrects over the global 195-cell family (135 initial plus 60 extension cells).

Rank-eligibility existence witness (Section 8.1.1).

```
python paper/src/make-witness.py
  -> paper/evidence/witness.json, paper/figures/witness.pdf
```

The command replays each seed one bar ahead to recover the true next-bar return (asserting that the canonical tape is policy-invariant), mixes it with a standardized noise path reused at every strength within that path (common random numbers), scores `sign_follow` and `deadband_hold` through `score_run` on every seed and on the pooled series, and first checks eligibility is monotone over the 13-point coarse grid. It bisects only a monotone first crossing, to a resolution of 0.005 in s, on the gap-band witness subset and the F1 table band. The whole procedure is repeated under five independent noise paths per cell, with noise-seed uniqueness asserted in the producer; the first path is serialized as the primary witness and all five are serialized under `noise_replicates`, which is what Table 10 summarizes. The JSON records every strength's per-seed PSR, pooled DSR, bootstrap p-value, eligibility flag, monotonicity check and every bracketed crossing.

Sealed evaluation seeds (Section 7.4.1).

```
python paper/src/make-sealed-seeds.py
  -> paper/evidence/sealed-seeds.json
```

The command derives 16 named slots publicly (the eight committed offsets plus eight drawn from the same 2^{16} band) and sealed (the same slots under a 32-byte salt from the operating system, committed by SHA-256 before the run), re-runs the predictability probe's band-scan adversary unchanged against both, then reveals the salt and replays the sealed scenarios against the recomputed seeds. The JSON records the commitment, the revealed salt, the table-build time and every per-trial recovery and replay record.

F6: adverse-selection markouts (Section 7.3).

```
python paper/src/make-f6-adverse-selection.py
  -> paper/evidence/f6-adverse-selection.json,
     paper/figures/f6-markouts.pdf, paper/figures/f6-endogenous.pdf
```

The same command runs the endogenous arm and its six-value impact sweep, serializes them under the `endogenous` key and writes `paper/figures/f6-endogenous.pdf` (Section 7.3.1).

F7: failure-mode distributions (Section 8.4).

```
python paper/src/make-f7-failures.py
  -> paper/evidence/f7-failures.json, paper/figures/f7-failures.pdf
```

F8: ecology under shocks, 8 seeds per schedule (Section 8.5).

```
python paper/src/make-f8-ecology.py
  -> paper/evidence/f8-ecology.json,
     paper/figures/f8-ecology-control.pdf,
     paper/figures/f8-ecology-shocked.pdf
```

The evidence carries the full seed-0 reports (the figures' source) plus, under `multi_seed`, per-seed winners, outcome counts and the cross-seed winner distributions behind Table 13.

Generator predictability (Section 7.4).

```
python paper/src/make-predictability.py
  -> paper/evidence/predictability.json,
     paper/figures/predictability.pdf
```

The command evaluates baseline, causal AR(5) and true-seed oracle predictors on 16 seeds per tier, then times exhaustive recovery in a 2^{16} candidate band. The JSON contains every per-seed score and the measured search timing.

Throughput and compute cost (Section 6.6).

```
python paper/src/make-throughput.py
  -> paper/evidence/throughput.json
```

The command times the Python surface directly (scenario generation, Gymnasium-adapter stepping, and one full six-policy evaluation over the canonical held-out band on all three tiers), then shells out to the native and WebAssembly benchmarks and parses their output, so all three rows of Table 3 come from one invocation. The two sub-benchmarks also run standalone:

```
cargo run --release -p sharpearena --example bench-steps
node npm/sharpearena/bench/throughput.js
```

Every rate is the median of three repeats of 500 episodes on a 4×120 panel, after a warm-up invocation that is discarded; the evidence JSON records every individual repeat and the host's platform string, so a reader can see the spread rather than only the median. Throughput is hardware-dependent and is the one quantity in this paper that a replicator should expect to differ.

Dispersion layers. The F2, F3, F5 and F6 scripts serialize the per-episode or per-seed vectors behind every reported interval into their evidence JSONs (`per_episode_regret`, `per_seed_returns` with `band_dsr_ci` and `transfer_gap_ci`, `dispersion`, and `per_episode_markout_per_unit` with `gap_stats` respectively), each with its CI convention recorded in the file, so every interval in Sections 7 and 8 is recomputable from the committed evidence.

Regenerating every figure from the committed evidence.

```
python paper/src/make-figures.py
```

Each `make-f*` script produces its own figure at run time; `make-figures.py` re-renders all figures from the JSON already in `paper/evidence/` without re-running any experiment, so a reader can check that no figure contains a number the evidence does not.